

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN I



BÁO CÁO ĐỒ ÁN CUỐI KỲ
CÁC HỆ THỐNG PHÂN TÁN

Đề tài: Phát triển hệ thống chat ngang hàng P2P
(Peer-to-Peer Chat System)

Giảng viên: Kim Ngọc Bách

Nhóm lớp môn học: 02

Nhóm bài tập lớn: 19

Sinh viên thực hiện:

Nguyễn Danh Toàn – B22DCCN740

Trần Tuấn Quỳnh – B22DCCN680

Văn Nhật Minh – B22DCCN548

Hà Nội, tháng 5 năm 2026

LỜI CẢM ƠN

Lời đầu tiên cho phép nhóm em gửi lời cảm ơn sâu sắc tới thầy Kim Ngọc Bách, người đã tận tình hướng dẫn em trong quá trình thực hiện môn học các hệ thống phân tán. Với sự dẫn dắt, những lời khuyên quý báu và sự tận tâm của thầy, em đã có cơ hội tiếp cận và hoàn thành đề tài "Phát triển hệ thống chat ngang hàng P2P" một cách hiệu quả. Những góp ý chi tiết và kiến thức chuyên môn từ thầy đã giúp em không chỉ hiểu sâu hơn về các khía cạnh kỹ thuật, mà còn nhận thức được tầm quan trọng của tư duy logic và phương pháp làm việc khoa học.

Một lần nữa, em xin chân thành cảm ơn thầy vì sự tận tình và tâm huyết trong việc hướng dẫn em cũng như các bạn sinh viên trong học tập và nghiên cứu. Em hy vọng rằng trong tương lai sẽ tiếp tục nhận được sự hỗ trợ và chỉ bảo từ thầy để hoàn thiện bản thân hơn nữa.

MỤC LỤC

MỤC LỤC	2
DANH MỤC HÌNH ẢNH.....	4
LỜI MỞ ĐẦU	5
CHƯƠNG 1. TỔNG QUAN VỀ HỆ THỐNG CHAT NGANG HÀNG P2P	6
1.1 Giới thiệu chung về đề tài	6
1.2 Lý do chọn đề tài	7
1.3 Mục tiêu của đề tài	8
1.4 Ý nghĩa thực tiễn	9
1.5 Cấu trúc dự án hệ thống	10
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ PHÂN TÍCH THIẾT KẾ HỆ THỐNG CHAT NGANG HÀNG P2P	12
2.1 Cơ sở lý thuyết	12
2.1.1 Mạng ngang hàng P2P.....	12
2.1.2 Giao thức truyền thông TCP/IP và cơ chế Socket.....	13
2.1.3 Cơ chế Length-Prefix Framing.....	15
2.2 Phân tích yêu cầu hệ thống.....	17
2.2.1 Yêu cầu chức năng	17
2.2.2 Yêu cầu phi chức năng	18
2.3 Thiết kế kiến trúc hệ thống.....	20
2.4 Thiết kế cơ sở dữ liệu	21
CHƯƠNG 3: THỰC NGHIỆM, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	26
3.1 Môi trường triển khai và công nghệ sử dụng	26
3.2 Các cơ chế cốt lõi của hệ thống.....	26
3.2.1 Cơ chế Peer Discovery và quản lý trạng thái (HeartBeat)	26

3.2.2 Cơ chế Tin nhắn ngoại tuyến.....	27
3.2.3 Cơ chế đồng bộ trạng thái gõ chữ và đã đọc	28
3.2.4 Cơ chế Trích dẫn trả lời & Chuyển tiếp tin nhắn	30
3.2.5 Cơ chế Bảo mật & Mã hóa gói tin.....	30
3.3 Giao diện hệ thống	31
3.4 Kịch bản kiểm thử và sửa lỗi.....	46
3.4.1 Mất kết nối đột ngột	46
3.4.2 Gửi tin nhắn cho người dùng ngoại tuyến.....	47
3.4.3 Đăng ký trùng tên tài khoản	48
3.5 Đánh giá và hướng phát triển	50
3.5.1. Ưu điểm của hệ thống	50
3.5.2. Hạn chế thực tế của hệ thống	50
3.5.3. Hướng phát triển tương lai	51
KẾT LUẬN	52
TÀI LIỆU THAM KHẢO	53

DANH MỤC HÌNH ẢNH

Hình 2.1 So sánh mô hình Client-Server với P2P.....	12
Hình 2.2 Đặc tính hướng kết nối.....	14
Hình 2.3 Hiện tượng dính gói và phân mảnh gói tin.....	16
Hình 2.4 Khung cấu trúc gói tin gửi đi trên đường truyền.....	16
Hình 2.5 Sơ đồ kiến trúc hệ thống.....	20
Hình 2.6 Sơ đồ liên kết cơ sở dữ liệu.....	25
Hình 3.1 Cơ chế quản lý trạng thái	26
Hình 3.2 Cơ chế lưu trữ và chuyển tiếp	27
Hình 3.3 Cơ chế đồng bộ trạng thái gõ chữ và đã đọc	29
Hình 3.4 Kết quả Console của Bootstrap Server.....	46
Hình 3.5 Kết quả trên Peer B	46
Hình 3.6 Peer A gửi 3 tin nhắn cho Peer B (đang Offline)	47
Hình 3.7 Dữ liệu bảng offline_messages	48
Hình 3.8 Giao diện khi Peer B online (Trạng thái chuyển sang Đã nhận).....	48
Hình 3.9 Bảng dữ liệu users đã tồn tại	49
Hình 3.10 Kết quả đăng ký bị lỗi khi trùng.....	49

LỜI MỞ ĐẦU

Trong kỷ nguyên số hóa và sự bùng nổ của Internet toàn cầu, nhu cầu trao đổi thông tin liên lạc trực tuyến thời gian thực đã trở thành một phần thiết yếu trong công việc và đời sống hàng ngày. Các ứng dụng nhắn tin hiện nay không chỉ yêu cầu tốc độ truyền tải nhanh chóng, giao diện thân thiện mà còn đòi hỏi tính bảo mật, độ tin cậy cao và khả năng bảo vệ quyền riêng tư dữ liệu của người dùng.

Hầu hết các hệ thống chat phổ biến hiện nay đều được xây dựng dựa trên kiến trúc Máy khách – Máy chủ tập trung Client-Server. Mặc dù mô hình này mang lại ưu điểm lớn về mặt quản lý dữ liệu dễ dàng, song nó lại tồn tại những hạn chế nhất định như: nguy cơ nghẽn mạng tại máy chủ trung tâm khi lượng người dùng tăng đột biến, chi phí duy trì hạ tầng máy chủ lớn và các mối lo ngại về quyền riêng tư khi toàn bộ nội dung hội thoại đều đi qua và lưu trữ tại bên thứ ba.

Để giải quyết các thách thức trên, kiến trúc Mạng ngang hàng Peer-to-Peer - P2P nổi lên như một giải pháp thay thế đầy tiềm năng. Trong mạng P2P, các thiết bị tham gia Peer vừa đóng vai trò là máy khách Client vừa là máy chủ Server, cho phép truyền tải dữ liệu trực tiếp với nhau mà không cần phụ thuộc hoàn toàn vào máy chủ trung tâm.

Nhận thức được tầm quan trọng và ưu điểm vượt trội đó, em đã lựa chọn đề tài: "Thiết kế và xây dựng Hệ thống Chat ngang hàng P2P sử dụng Java Socket và cơ sở dữ liệu MySQL". Hệ thống được thiết kế theo mô hình P2P, kết hợp sức mạnh điều phối, quản lý đăng ký của máy chủ trung tâm Bootstrap Server với khả năng truyền thông trực tiếp, hiệu quả của các nút mạng ngang hàng.

CHƯƠNG 1. TỔNG QUAN VỀ HỆ THỐNG CHAT NGANG HÀNG P2P

1.1 Giới thiệu chung về đề tài

Trong thời đại kinh tế số và giao tiếp trực tuyến phát triển mạnh mẽ như hiện nay, phương thức liên lạc và trao đổi thông tin đang chứng kiến những bước chuyển đổi sâu sắc. Công nghệ không chỉ còn là công cụ nhắn tin hay gọi điện thuần túy, mà đang dần trở thành một phần hữu cơ trong việc tối ưu hóa hiệu suất làm việc, kết nối cộng đồng và bảo vệ quyền riêng tư dữ liệu cá nhân. Từ nhu cầu đó, hệ thống trò chuyện thời gian thực thông minh ra đời, giúp người dùng quản lý cuộc hội thoại, tương tác trực tiếp và chia sẻ thông tin một cách chủ động và an toàn hơn.

Với đề tài "Hệ thống chat ngang hàng P2P", nhóm chúng em mong muốn xây dựng một mô hình giao tiếp có khả năng tối ưu hóa đường truyền và bảo mật thông tin phù hợp. Ở đây, hệ thống đóng vai trò như một cầu nối thông minh — tự động định vị, thiết lập kết nối và chuyển tải dữ liệu trực tiếp giữa các thiết bị người dùng theo các yêu cầu giao tiếp thời gian thực đã định, thay vì để người dùng phải phụ thuộc hoàn toàn vào băng thông của máy chủ trung tâm. Điều này giúp người dùng dễ dàng trao đổi tin nhắn, tệp tin đúng theo tiêu chuẩn thực tế, đồng thời mang lại trải nghiệm vận hành hiện đại, chính xác và nhất quán.

Cụ thể, thông tin tương tác (như trạng thái hoạt động, tin nhắn văn bản, tệp tin truyền tải) sẽ được hệ thống tiếp nhận. Hệ thống sẽ phân tích các đặc điểm kết nối (địa chỉ IP, cổng TCP, nhịp tim heartbeat), nhận dạng đúng chủng loại thông tin cần xử lý (tin nhắn cá nhân, tin nhắn nhóm, yêu cầu kết bạn), rồi hiển thị kết quả và tự động sắp xếp vào đúng các danh mục hội thoại mà người dùng quan tâm.

Về mặt công nghệ, hệ thống được xây dựng dựa trên sự kết hợp giữa giao thức mạng TCP Socket và cơ sở dữ liệu MySQL thông qua bộ quản lý kết nối tập trung HikariCP. Mỗi thông điệp trên hệ thống được xem như một đối tượng JSON chuẩn hóa, cho phép hệ thống giao tiếp hiệu quả qua cơ chế đóng gói tiền tố độ dài. Khi người dùng tương

tác, hệ thống sẽ xử lý thông tin hành vi (như chỉ báo đang gõ chữ, xác nhận đã đọc) và đưa ra các phản hồi tương ứng ngay lập tức.

Điểm nổi bật của hệ thống là không cần người dùng phải cấu hình hay thiết lập địa chỉ kết nối thủ công cho từng người bạn, chỉ cần đăng nhập là mọi tiến trình kết nối ngang hàng và chuyển tiếp tin nhắn ngoại tuyến sẽ được thực hiện hoàn toàn tự động. Bên cạnh sự chính xác, hệ thống còn mang lại nhiều lợi ích khác như tiết kiệm tài nguyên băng thông máy chủ (nhờ việc truyền tải trực tiếp giữa các peer), nâng cao chất lượng trải nghiệm đồng nhất với giao diện tối màu hiện đại. Ngoài ra, việc lưu trữ và phân loại tin nhắn chính xác giúp hạn chế việc người dùng tiếp cận nhầm lẫn giữa các luồng hội thoại cá nhân và hội thoại nhóm.

Tóm lại, "Hệ thống chat ngang hàng P2P" không chỉ giúp chúng ta kết nối và trao đổi thông tin một cách nhanh chóng, chính xác mà còn là một bước tiến quan trọng trong việc ứng dụng công nghệ mạng ngang hàng vào đời sống giao tiếp số. Thay vì thao tác thủ công phức tạp hay phụ thuộc hoàn toàn vào dịch vụ lưu trữ tập trung của bên thứ ba, giờ đây người dùng chỉ cần đăng nhập và hệ thống sẽ thực hiện phần còn lại. Đây chính là hướng phát triển tất yếu của công nghệ truyền thông trong tương lai, nơi con người và các nút mạng có thể tương tác tự nhiên để cùng nhau tạo nên một chuỗi trao đổi nội dung hiện đại, hiệu quả và bảo mật hơn.

1.2 Lý do chọn đề tài

Hiện nay, phần lớn việc kết nối và lưu trữ dữ liệu trò chuyện của người dùng vẫn chủ yếu phụ thuộc vào mô hình máy chủ tập trung Client-Server. Điều này vừa tiềm ẩn nguy cơ mất an toàn thông tin (khi toàn bộ nội dung hội thoại bị kiểm soát bởi một bên thứ ba), vừa dễ gây ra tình trạng quá tải hoặc gián đoạn dịch vụ tại máy chủ trung tâm trước sự bùng nổ quá lớn của lưu lượng truy cập thời gian thực, khiến người dùng khó tối ưu hóa tốc độ truyền tải cũng như tính riêng tư của dữ liệu hội thoại của mình.

Chính vì vậy, nhóm chúng em lựa chọn đề tài "Hệ thống chat ngang hàng P2P" với mong muốn xây dựng một mô hình tự động, đơn giản, dễ vận hành, có khả năng kết nối trực tiếp các nút mạng thông qua việc định vị địa chỉ động từ máy chủ điều phối. Hệ thống chat ngang hàng sẽ giúp nhận diện trạng thái hoạt động của từng người dùng và

điều khiển việc truyền gửi tin nhắn, tệp tin một cách chính xác, nhanh chóng và bảo mật hơn.

Ngoài tính ứng dụng thực tế, đề tài còn giúp nhóm được tiếp cận và vận dụng nhiều kiến thức công nghệ như lập trình socket mạng Java Socket, quản lý cơ sở dữ liệu MySQL kết hợp HikariCP và ứng dụng lập trình giao diện Java Swing với FlatLaf, qua đó nâng cao kỹ năng chuyên môn. Bên cạnh đó, hệ thống còn mang ý nghĩa về mặt trải nghiệm khi có thể hỗ trợ các nền tảng truyền thông nâng cao chất lượng dịch vụ đầu ra (thông qua cơ chế lưu tin nhắn offline và đồng bộ trạng thái đã đọc), giảm sự phụ thuộc băng thông vào máy chủ trung tâm và tăng khả năng tương tác bảo mật giữa người dùng với người dùng.

Tóm lại, nhóm chọn đề tài này vì nó vừa thiết thực, hiện đại, có khả năng triển khai cao, vừa phù hợp với xu hướng phát triển công nghệ tự động hóa và bảo mật dữ liệu trong lĩnh vực truyền thông số hiện nay.

1.3 Mục tiêu của đề tài

Mục tiêu chính của đề tài "Hệ thống chat ngang hàng P2P" là xây dựng một mô hình liên lạc thời gian thực có khả năng truyền dẫn dữ liệu (tin nhắn văn bản, trạng thái kết nối) trực tiếp giữa các thiết bị người dùng một cách nhanh chóng, an toàn và riêng tư thông qua lập trình socket mạng và giao thức đóng gói dữ liệu tự định nghĩa.

Cụ thể hơn, hệ thống hướng đến việc cho phép tiếp nhận thông tin đầu vào từ người dùng (như thông tin đăng ký/đăng nhập, nội dung tin nhắn chat cá nhân hoặc chat nhóm, tệp tin chia sẻ), bộ xử lý luồng mạng sẽ tự động thiết lập và duy trì kết nối TCP Socket trực tiếp giữa người gửi và người nhận. Hệ thống sẽ đảm nhiệm việc phân tích trạng thái hoạt động online/offline, nhận diện trạng thái gõ chữ, xác nhận đã đọc và gửi phản hồi kết quả hiển thị đến giao diện người dùng để thực hiện đúng yêu cầu kết nối thời gian thực.

Bên cạnh đó, đề tài còn nhằm nâng cao khả năng ứng dụng công nghệ lập trình mạng đa luồng vào đời sống thực tế, giúp các nền tảng chat doanh nghiệp hoặc nội bộ trải nghiệm một quy trình kết nối tự động, hiệu quả và bảo mật hơn. Đồng thời, hệ thống cũng góp phần tiết kiệm tài nguyên băng thông cho máy chủ trung tâm nhờ cơ chế chia

sẽ tải trực tiếp, giảm thiểu rủi ro rò rỉ thông tin trên các kênh trung gian và nâng cao chất lượng trải nghiệm hội thoại đồng nhất cho người dùng.

Ngoài mục tiêu kỹ thuật, nhóm còn đặt mục tiêu học tập là rèn luyện kỹ năng nghiên cứu, lập trình và triển khai hệ thống thực tế, giúp nắm vững kiến thức về phần mềm xử lý luồng dữ liệu TCP, thiết kế giao diện Swing tùy biến, tối ưu hóa truy vấn cơ sở dữ liệu MySQL và kỹ thuật quản lý đa luồng đồng thời.

Tóm lại, mục tiêu cuối cùng của đề tài là tạo ra một mô hình hệ thống chat bảo mật thông minh có tính ứng dụng cao, dễ mở rộng có thể tích hợp thêm các thuật toán mã hóa đầu cuối RSA/AES hoặc cơ chế truyền file đa phân đoạn và phù hợp với điều kiện thực tế tại Việt Nam, góp phần thúc đẩy xu hướng phi tập trung hóa trong lĩnh vực dịch vụ số và trao đổi thông tin trực tuyến.

1.4 Ý nghĩa thực tiễn

Đầu tiên về mặt thực tế, hệ thống này giúp các nền tảng truyền thông nội bộ và người dùng cuối quản lý, tiếp cận luồng trao đổi thông tin một cách tiện lợi, nhanh chóng và bảo mật hơn. Thay vì phải cấu hình địa chỉ IP tĩnh và kết nối thủ công phức tạp, hệ thống chỉ cần dựa trên danh mục bạn bè và cơ chế đăng ký tự động của máy chủ điều phối để nhận diện và thiết lập đường truyền trực tiếp phù hợp. Điều này giúp tiết kiệm thời gian, giảm các thao tác lặp lại và mang lại chất lượng trải nghiệm hội thoại đồng nhất. Trong trường hợp người dùng nhận tin nhắn đang ngoại tuyến, hệ thống sẽ tự động chuyển sang chế độ lưu trữ tin nhắn ngoại tuyến tại cơ sở dữ liệu trung tâm để đảm bảo luồng thông tin liên lạc luôn xuyên suốt và không bị thất lạc.

Cơ chế truyền thông tin ngang hàng: Sử dụng các kết nối socket TCP trực tiếp kết hợp kỹ thuật đóng gói tiền tố độ dài để so sánh và kiểm soát chính xác các gói tin dựa trên cấu trúc JSON. Điều này cho phép hệ thống tìm ra đường truyền trực tiếp tối ưu nhất, truyền dữ liệu thời gian thực giữa hai thiết bị mà không cần đi qua băng thông trung gian của máy chủ, nâng cao tốc độ phản hồi và bảo mật tối đa nội dung hội thoại.

Hệ thống giám sát và đồng bộ trạng thái: Được sử dụng để kết nối với cơ sở dữ liệu người dùng, liên tục kiểm tra tín hiệu hoạt động và cập nhật đồng bộ danh sách bạn bè online/offline trực quan. Cơ chế này cũng hỗ trợ lưu trữ thông tin về lịch sử tương tác

(như trạng thái đang gõ và xác nhận đã đọc) để liên tục tối ưu hóa độ chính xác của trạng thái hiển thị trong các phiên làm việc sau.

Ngoài ra, việc áp dụng công nghệ mạng ngang hàng P2P trong lĩnh vực truyền thông còn giúp nâng cao tính chủ động và thông minh của các ứng dụng trao đổi dữ liệu, góp phần vào xu hướng chuyển đổi số và xây dựng nền công nghệ dịch vụ mạng an toàn tại Việt Nam. Hệ thống cũng có thể mở rộng, cập nhật thêm nhiều giao thức bảo mật (như mã hóa đầu cuối E2EE bằng cặp khóa RSA/AES) và thuộc tính truyền tệp tin phân đoạn khác nhau, từ đó hướng tới một môi trường làm việc hiện đại, hiệu quả và giảm thiểu tỉ lệ rò rỉ dữ liệu do các cuộc tấn công mạng trung gian.

Còn về mặt giáo dục và nghiên cứu, đề tài giúp nhóm thực hiện có cơ hội ứng dụng kiến thức đã học vào thực tế, kết hợp giữa lập trình socket mạng đa luồng, quản lý cơ sở dữ liệu tối ưu với HikariCP và lập trình giao diện người dùng Swing. Qua đó, nhóm có thể nâng cao kỹ năng về thiết kế hệ thống, xử lý luồng dữ liệu mạng đồng thời và lập trình phân mềm đồng bộ.

Tóm lại, ý nghĩa thực tiễn của đề tài không chỉ nằm ở việc tạo ra một mô hình chat trực tuyến tự động, mà còn ở việc góp phần đưa công nghệ truyền thông phi tập trung đến gần hơn với đời sống và công việc hàng ngày, hướng tới một tương lai có thể dễ dàng sử dụng công nghệ để nâng cao chất lượng và giá trị bảo mật của các dịch vụ nội dung số.

1.5 Cấu trúc dự án hệ thống

Hệ thống nhắn tin thời gian thực ngang hàng bao gồm các thành phần chính sau:

Khối điều phối trung tâm Bootstrap Registry: Sử dụng các luồng socket TCP để đăng ký và quản lý địa chỉ mạng (IP, Port) của các thiết bị client khi họ kết nối. Từ đó, hệ thống xác định trạng thái sống/chết của các nút mạng để phát đi các cập nhật trạng thái hoạt động đến danh sách liên hệ của người dùng khác một cách nhanh chóng.

Khối truyền thông ngang hàng Peer Node Core: Sử dụng công nghệ thiết lập socket cục bộ và quản lý đa luồng tại mỗi client. Khối này đảm nhiệm việc kết nối đến máy chủ trung tâm và kết nối trực tiếp đến các peer khác khi cần gửi dữ liệu, tự động kích hoạt các hàm phản hồi sự kiện Listeners/Callbacks để tiếp nhận và phản hồi tin

nhấn ngay lập tức.

Bộ điều phối tin nhắn lai Hybrid Message Controller: Đây là thành phần trung tâm đóng vai trò kết nối và điều phối. Bộ điều phối này sẽ linh hoạt kết hợp dữ liệu giữa truyền tin trực tiếp khi đối tác online và chuyển hướng lưu trữ trung gian khi đối tác offline. Trong trường hợp người nhận đang ngoại tuyến, hệ thống sẽ tự động chuyển sang chế độ lưu trữ tin nhắn ngoại tuyến tại database tập trung để đảm bảo luồng tin nhắn luôn được duy trì xuyên suốt.

Giao thức đóng gói và phân mảnh Protocol Framing Engine: Sử dụng các công thức tính toán và xử lý mảng byte như chuyển đổi độ dài 32-bit Big-endian đầu gói tin và phân tích cú pháp JSON để đóng gói thông điệp. Điều này cho phép hệ thống đọc/ghi luồng nhị phân an toàn trên TCP, tránh lỗi dính gói hoặc chia cắt gói tin khi truyền dữ liệu tốc độ cao.

Giao diện người dùng và Quản lý cơ sở dữ liệu GUI & Data API: Được sử dụng để kết nối với cơ sở dữ liệu lưu trữ lịch sử chat, quan hệ bạn bè và cấu trúc nhóm. Giao diện được thiết kế bằng FlatLaf Dark theme hỗ trợ hiển thị danh sách hội thoại, các bong bóng chat tương ứng với từng cuộc trò chuyện, chỉ báo trạng thái gõ chữ, xác nhận đã đọc và lưu trữ thông tin lịch sử tương tác để liên tục tối ưu hóa độ chính xác cho các phiên sử dụng tiếp theo.

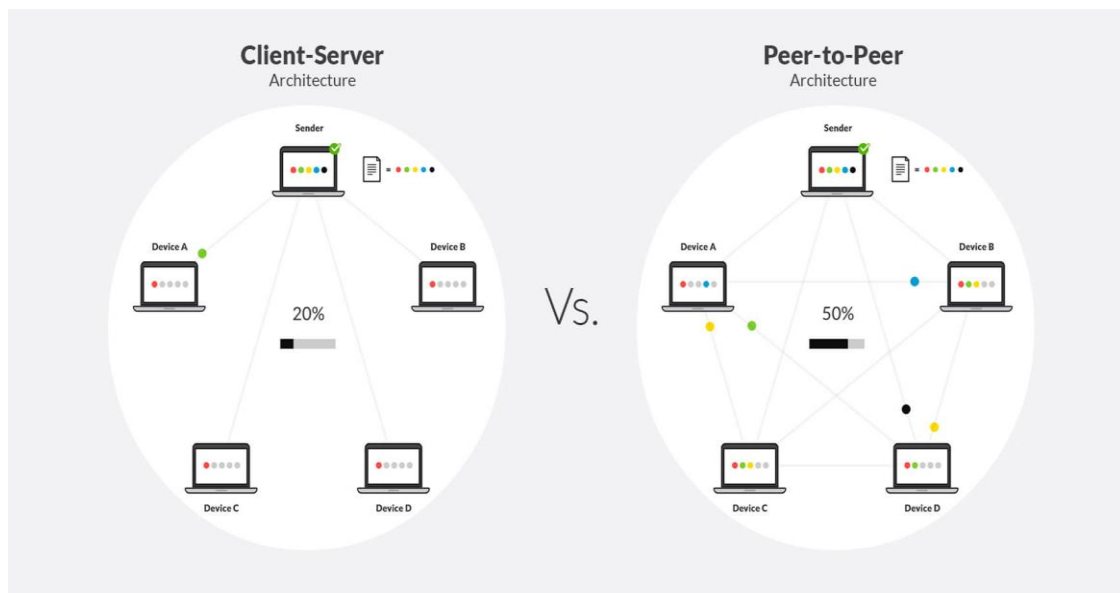
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ PHÂN TÍCH THIẾT KẾ HỆ THỐNG CHAT NGANG HÀNG P2P

2.1 Cơ sở lý thuyết

2.1.1 Mạng ngang hàng P2P

Mạng ngang hàng P2P là một kiến trúc mạng phân tán, trong đó các thiết bị tham gia vào hệ thống Peer chia sẻ một phần tài nguyên của chính chúng như năng lượng xử lý, băng thông đường truyền, bộ nhớ lưu trữ cho các nút mạng khác mà không cần thông qua các máy chủ điều phối trung tâm.

Khác với mô hình Client-Server truyền thống vốn phân định rõ ranh giới giữa bên yêu cầu dịch vụ Client và bên cung cấp dịch vụ Server, trong kiến trúc P2P, mỗi nút mạng vừa đóng vai trò là một Client (khi yêu cầu nhận dữ liệu từ nút khác) vừa đồng thời đóng vai trò là một Server (khi cung cấp dữ liệu cho các nút khác trong mạng).



Hình 2.1 So sánh mô hình Client-Server với P2P

Thách thức lớn nhất của mạng ngang hàng trong môi trường Internet thực tế là Định vị nút mạng (Peer Discovery). Do hầu hết người dùng cuối sử dụng mạng gia đình hoặc

di động với địa chỉ IP động Dynamic IP thay đổi liên tục sau mỗi lần reset mạng hoặc chuyển vùng kết nối, các peer không thể tự biết địa chỉ IP và Port hiện tại của nhau để thiết lập kết nối trực tiếp.

Bootstrap Server giải quyết bài toán này đóng vai trò như một danh bạ trung tâm Registry Directory:

Đăng ký địa chỉ động: Khi một Peer khởi động và đăng nhập thành công, nó gửi một thông điệp đăng ký lên Bootstrap Server kèm theo địa chỉ IP và số cổng TCP mà nó đang lắng nghe kết nối. Server sẽ ghi nhận các thông tin này vào cơ sở dữ liệu và vùng nhớ đệm PeerRegistry.

Duy trì danh bạ thời gian thực: Thông qua các tín hiệu nhịp tim định kỳ Heartbeat, Bootstrap Server kiểm tra xem IP/Port của peer đó còn khả dụng hay không. Nếu IP của peer thay đổi, gói tin Heartbeat mới gửi lên sẽ giúp Server cập nhật ngay lập tức địa chỉ IP mới vào danh bạ.

Cung cấp thông tin định vị: Khi Peer A muốn gửi tin nhắn đến Peer B, thay vì dò tìm mù quáng trên toàn mạng, Peer A chỉ cần truy vấn Bootstrap Server để lấy địa chỉ IP và Port hiện tại của Peer B, từ đó nhanh chóng chủ động thiết lập đường truyền TCP trực tiếp tới Peer B.

2.1.2 Giao thức truyền thông TCP/IP và cơ chế Socket

Trong mô hình tham chiếu giao thức TCP/IP, lớp Giao vận Transport Layer chịu trách nhiệm thiết lập luồng truyền thông trực tiếp giữa hai tiến trình ứng dụng đang chạy trên các máy chủ khác nhau. Ở lớp này, hai giao thức phổ biến nhất được sử dụng là TCP và UDP.

Đối với các ứng dụng nhắn tin văn bản thời gian thực Chat Application, độ chính xác của nội dung và thứ tự truyền tin là yếu tố quan trọng nhất. Do đó, giao thức TCP được lựa chọn làm nền tảng cốt lõi nhờ khả năng cung cấp luồng truyền tải dữ liệu có độ tin cậy cao, hướng kết nối và kiểm soát dòng dữ liệu một cách chặt chẽ.

Socket là một giao diện lập trình ứng dụng (API) đóng vai trò là một điểm cuối (Endpoint) của luồng truyền thông hai chiều giữa hai chương trình chạy trên mạng. Một Socket được định danh duy nhất bởi sự kết hợp của:

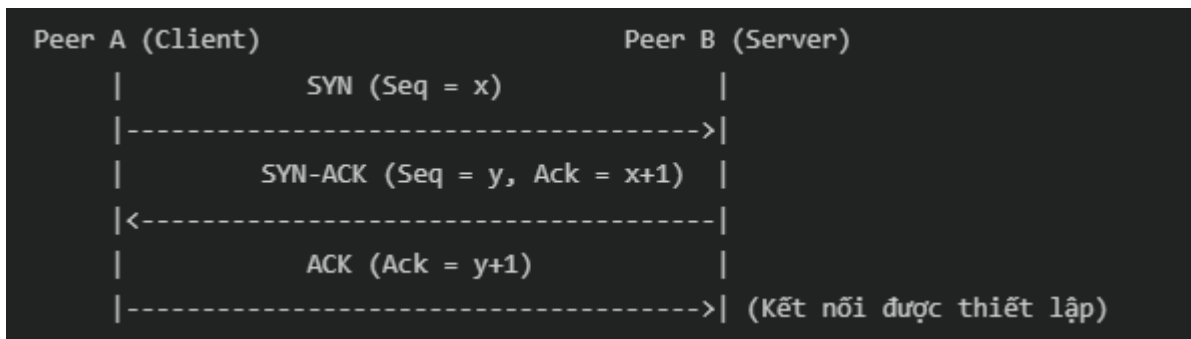
Socket = <Địa chỉ IP, Số hiệu cổng Port>

Socket phía Máy chủ ServerSocket: Chờ đợi (Listen) các yêu cầu kết nối từ các máy khách tại một cổng dịch vụ cố định (ví dụ: Bootstrap Server lắng nghe ở cổng 9000).

Socket phía Máy khách Socket: Chủ động thiết lập kết nối tới cổng dịch vụ của Server. Khi kết nối được thiết lập thành công, cả hai bên sẽ sở hữu một cặp luồng vào/ra (InputStream và OutputStream) để thực hiện đọc/ghi dữ liệu thô.

Một kết nối Socket hoạt động trên nền tảng TCP sở hữu các cơ chế kỹ thuật giúp đảm bảo tính vẹn toàn dữ liệu như sau:

Đặc tính hướng kết nối: Trước khi truyền bất kỳ dữ liệu thực tế nào, TCP yêu cầu hai tiến trình phải thiết lập kết nối logic với nhau thông qua quá trình Bắt tay ba bước Three-way Handshake. Quá trình này giúp đồng bộ hóa các số thứ tự gói tin Sequence Number và cấp phát các vùng đệm bộ nhớ cần thiết trên cả hai thiết bị.



Hình 2.2 Đặc tính hướng kết nối

Đảm bảo độ tin cậy và không mất gói tin:

Cơ chế báo nhận ACK: Khi nhận được một gói tin, bên nhận có nhiệm vụ gửi lại một gói tin báo nhận Acknowledgment - ACK để xác nhận đã nhận dữ liệu thành công.

Cơ chế truyền lại tự động ARQ - Automatic Repeat Request: Nếu bên gửi không nhận được ACK trong một khoảng thời gian chờ nhất định, hoặc nhận được các ACK trùng lặp cảnh báo mất gói, nó sẽ tự động gửi lại gói dữ liệu đó. Điều này đảm bảo dữ liệu không bị thất lạc trên đường truyền vật lý.

Đảm bảo đúng thứ tự dữ liệu: TCP đánh số thứ cho từng byte dữ liệu được truyền

đi. Phía nhận sẽ sử dụng các số thứ tự này để tái cấu trúc lại thông điệp theo đúng trình tự ban đầu kể cả khi các gói tin truyền qua các tuyến đường mạng khác nhau và đến đích lệch thời gian.

Kiểm soát luồng và chống tắc nghẽn: TCP sử dụng cơ chế cửa sổ trượt Sliding Window để điều phối lượng dữ liệu gửi đi sao cho phù hợp với khả năng tiếp nhận của vùng đệm phía nhận và tình trạng chịu tải của mạng, ngăn ngừa việc tràn bộ đệm gây mất dữ liệu.

Trong hệ thống Chat ngang hàng, cơ chế Socket cho phép:

Duy trì trạng thái kết nối liên tục: Giúp các tin nhắn truyền đi ngay lập tức với độ trễ thấp thay vì phải liên tục tạo yêu cầu kết nối mới giống như giao thức HTTP Web.

Truyền tải hai chiều đồng thời: Giúp mỗi Peer có thể vừa gửi tin nhắn đi, vừa sẵn sàng nhận tin nhắn đến tại cùng một thời điểm mà không gây nghẽn tiến trình của giao diện người dùng.

2.1.3 Cơ chế Length-Prefix Framing

Một trong những đặc trưng quan trọng nhất của giao thức TCP là tính chất hướng luồng. Điều này có nghĩa là TCP coi dữ liệu truyền tải qua Socket là một chuỗi luồng các byte liên tục, không có ranh giới tự nhiên hay ranh giới gói tin. TCP không giữ lại cấu trúc thông điệp gốc khi truyền qua tầng vật lý; nó chỉ đảm bảo truyền nhận đủ các byte và đúng thứ tự.

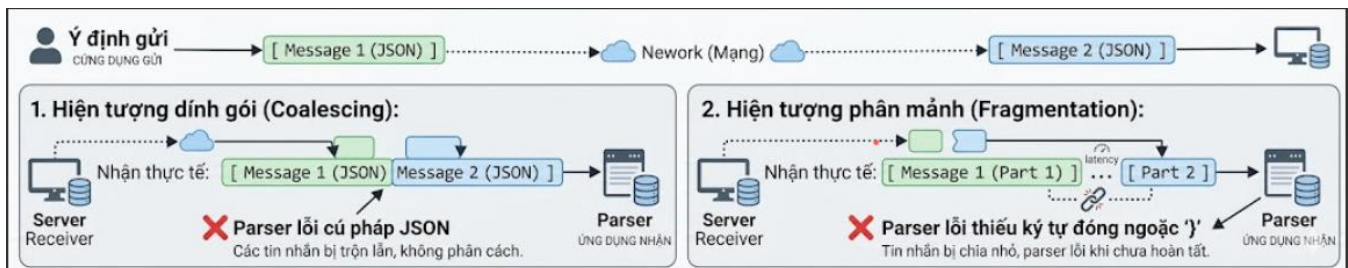
Khi một ứng dụng gửi chuỗi ký tự JSON qua TCP Socket, tầng truyền vận TCP sẽ tự ý gom hoặc chia nhỏ mảng byte của chuỗi JSON này tùy thuộc vào kích thước của bộ đệm gửi/nhận và kích thước đơn vị truyền dẫn tối đa của mạng MTU - Maximum Transmission Unit.

Do tính chất hướng luồng trên, hệ thống giao tiếp qua TCP Socket thường gặp phải hai vấn đề lớn sau khi cố gắng phân tích chuỗi JSON

Hiện tượng dính gói: Xảy ra khi bên gửi truyền các thông điệp nhỏ liên tiếp quá nhanh. TCP sẽ gom các byte của các thông điệp này lại và gửi đi trong cùng một phân đoạn mạng. Kết quả là bên nhận đọc được một chuỗi byte thô chứa nhiều chuỗi JSON dính liền với nhau (ví dụ: `{"type": "MSG", "content": "Hi"} {"type": "MSG", "content": "Bye"}`). Bộ phân

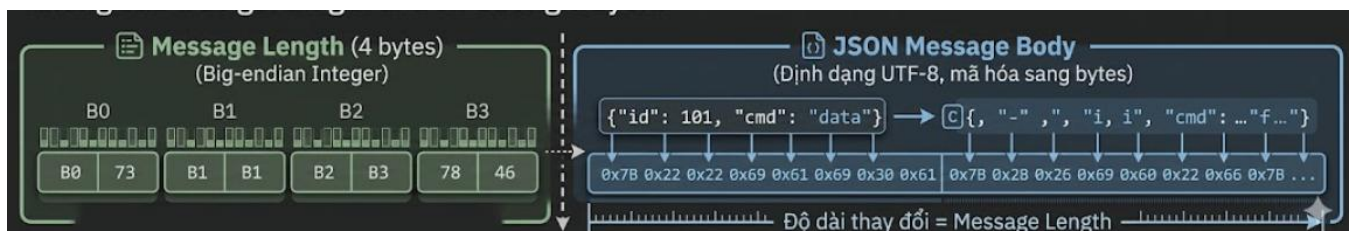
tích cú pháp JSON sẽ báo lỗi cú pháp ngay lập tức do chuỗi đầu vào không phải là một đối tượng JSON hợp lệ duy nhất.

Hiện tượng phân mảnh gói tin: Xảy ra khi thông điệp gửi đi có kích thước lớn hơn dung lượng trống trong bộ đệm mạng hoặc vượt quá chỉ số MTU của đường truyền. Khi đó, TCP sẽ tự động chia cắt chuỗi JSON thành nhiều phân đoạn nhỏ hơn và gửi đi qua nhiều gói tin IP. Phía nhận khi thực hiện lệnh đọc sẽ chỉ nhận được một phần của chuỗi JSON gốc (ví dụ: `{"type": "MSG", "content": "H"}`), dẫn đến lỗi không thể phân tích cú pháp do thiếu các ký tự đóng ngoặc nhọn.



Hình 2.3 Hiện tượng dính gói và phân mảnh gói tin

Để giải quyết triệt để hai lỗi trên và khôi phục lại ranh giới nguyên bản của từng thông điệp, hệ thống triển khai cơ chế Length-Prefix Framing (Phân khung dữ liệu bằng tiền tố độ dài). Ý tưởng cốt lõi của giải pháp này là: Trước khi truyền bất kỳ chuỗi dữ liệu JSON nào qua luồng mạng, ứng dụng sẽ tính toán độ dài kích thước số bytes của chuỗi JSON đó và gửi thông tin độ dài này đi trước dưới dạng một số nguyên có kích thước cố định là 4 bytes (32-bit Integer)



Hình 2.4 Khung cấu trúc gói tin gửi đi trên đường truyền

Quy trình xử lý cụ thể:

Phía gửi:

- Chuyển đổi đối tượng thông điệp Message thành chuỗi JSON.
- Mã hóa chuỗi JSON thành mảng byte sử dụng định dạng bảng mã UTF-8.
- Tính toán độ dài N của mảng byte này.
- Chuyển số nguyên N thành dạng nhị phân 4 bytes sử dụng thứ tự byte Big-endian (byte có ý nghĩa cao nhất được truyền đi trước) để đảm bảo tính độc lập với kiến trúc phần cứng của các hệ điều hành khác nhau.
- Gửi 4 bytes độ dài này qua Socket, sau đó gửi tiếp mảng byte dữ liệu JSON thực tế.

Phía nhận:

- Đầu tiên, hệ thống ép luồng đọc InputStream phải chờ và đọc đủ đúng 4 bytes từ Socket để giải mã ra giá trị số nguyên N đại diện cho độ dài phần thân tin nhắn.
- Sau khi đã xác định được chính xác kích thước N, hệ thống tạo một mảng byte tạm thời có kích thước đúng bằng N.
- Tiếp tục đọc từ Socket cho đến khi nhận đủ đúng N bytes tiếp theo. Hành động này giúp cô lập hoàn toàn phần dữ liệu của gói tin hiện tại, loại bỏ hiện tượng bị dính gói với tin nhắn sau hoặc thiếu hụt dữ liệu do phân mảnh.
- Chuyển mảng N bytes thu được thành chuỗi UTF-8 và tiến hành phân tích cú pháp JSON để nhận về đối tượng thông điệp hoàn chỉnh mà không gặp bất kỳ lỗi cú pháp nào.

2.2 Phân tích yêu cầu hệ thống

2.2.1 Yêu cầu chức năng

Đăng ký tài khoản mới: Cho phép người dùng mới tạo tài khoản bằng cách nhập Tên đăng nhập Username và Mật khẩu Password

Đăng nhập hệ thống: Xác thực danh tính người dùng khi truy cập ứng dụng

Tìm kiếm người dùng: Cho phép người dùng nhập từ khóa tìm kiếm để tìm các tài khoản khác trong hệ thống.

Quản lý lời mời kết bạn:

Gửi lời mời: Người dùng có thể gửi yêu cầu kết bạn đến tài khoản khác

Đồng ý/Từ chối: Phía người nhận có thể chọn "Đồng ý" hoặc "Từ chối".

Hủy kết bạn: Cho phép xóa mối quan hệ bạn bè trực tiếp thông qua menu chuột phải trên danh bạ

Nhắn tin trực tiếp: Người dùng có thể chọn một liên hệ trực tuyến hoặc ngoại tuyến trong danh sách để gửi tin nhắn văn bản.

Chỉ báo đang gõ chữ: Khi một bên đang nhập nội dung trong ô tin nhắn, phía bên kia sẽ nhận được thông điệp trạng thái và hiển thị dòng chữ thông báo nhấp nháy "[Tên người gửi] đang nhập...".

Xác nhận trạng thái tin nhắn: Tin nhắn sau khi gửi đi sẽ được hiển thị các trạng thái đồng bộ: Đã gửi, Đã nhận (khi đối phương online), Đã đọc.

Tạo nhóm chat mới: Người dùng có thể khởi tạo một nhóm chat bằng cách nhập Tên nhóm và chọn tối thiểu 2 thành viên từ danh sách bạn bè.

Nhắn tin nhóm: Cho phép mọi thành viên trong nhóm gửi tin nhắn chung.

Quản lý nhóm chat của Admin: Người tạo nhóm sẽ nắm đặc quyền Admin: Có chức năng “Xóa thành viên” và “Xóa nhóm”

Thành viên thông thường chỉ có quyền tự rời khỏi nhóm chat.

Lưu giữ thông điệp khi ngoại tuyến: Khi một Peer gửi tin nhắn hoặc tin nhắn nhóm tới một người nhận đang không kết nối với hệ thống (Offline), Bootstrap Server sẽ tự động chuyển hướng nội dung tin nhắn đó vào bảng lưu trữ tạm thời offline_messages.

Đồng bộ lại khi trực tuyến: Ngay khi người dùng đó đăng nhập trở lại hệ thống, Bootstrap Server sẽ tự động truy vấn toàn bộ tin nhắn ngoại tuyến dành cho người dùng đó, gửi trả lại đầy đủ theo đúng trình tự thời gian, và sau đó xóa các bản ghi này khỏi bảng tạm để giải phóng tài nguyên

2.2.2 Yêu cầu phi chức năng

Khả năng xử lý song song và đa luồng

Phía Bootstrap Server:

- Hệ thống sử dụng một nhóm luồng dịch vụ được quản lý bởi lớp ExecutorService của Java. Khi có kết nối TCP Socket mới từ một Peer, máy

chủ không xử lý tuần tự mà lập tức tạo ra một tiến trình con riêng biệt ClientHandler chạy độc lập trên một luồng mới.

- Điều này cho phép máy chủ trung tâm có thể tiếp nhận và duy trì kết nối đồng thời với hàng trăm Peer cùng lúc mà không làm gián đoạn hay chậm trễ khả năng phản hồi của hệ thống.

Phía Peer Node Client:

- Khi ứng dụng Client khởi động, luồng đồ họa chính của Java Swing chỉ chịu trách nhiệm vẽ giao diện và tiếp nhận tương tác chuột/bàn phím từ người dùng.
- Toàn bộ các tác vụ mạng như lắng nghe luồng Socket từ máy chủ, gửi tin nhắn, nhận tin nhắn, truyền tệp tin hoặc gửi tín hiệu heartbeat đều được đẩy xuống các luồng chạy ngầm chạy song song. Nhờ đó, giao diện người dùng luôn mượt mà, không xảy ra hiện tượng đơ cứng màn hình khi mạng gặp sự cố hoặc khi đang tải tệp tin lớn.

Quản lý và lưu trữ dữ liệu bền vững

Để đảm bảo lịch sử hội thoại, thông tin tài khoản và danh sách nhóm chat không bị mất khi ứng dụng tắt đi, hệ thống triển khai cơ chế lưu trữ dữ liệu bền vững:

Hệ quản trị cơ sở dữ liệu MySQL: Lưu trữ tập trung mọi dữ liệu nghiệp vụ quan trọng. Thiết kế các chỉ mục Indexes hợp lý trên các trường định danh như username, message_id, group_id giúp tối ưu hóa tốc độ truy vấn lịch sử tin nhắn cũ.

Bộ quản lý kết nối HikariCP: Việc mở và đóng kết nối đến MySQL liên tục cho mỗi tin nhắn chat sẽ gây hao phí tài nguyên máy chủ và tăng độ trễ đường truyền. Hệ thống sử dụng thư viện HikariCP để thiết lập một hồ chứa kết nối sẵn có. Các tiến trình xử lý luồng mạng sẽ mượn kết nối từ pool để thực hiện truy vấn SQL và hoàn trả lại ngay sau khi hoàn tất, giúp tối ưu hóa hiệu năng truy cập database của toàn hệ thống.

Giao diện người dùng trực quan, hiện đại (GUI)

Giao diện người dùng được thiết kế bằng ngôn ngữ Java Swing nhưng sở hữu diện mạo hiện đại nhằm tối ưu hóa trải nghiệm tương tác:

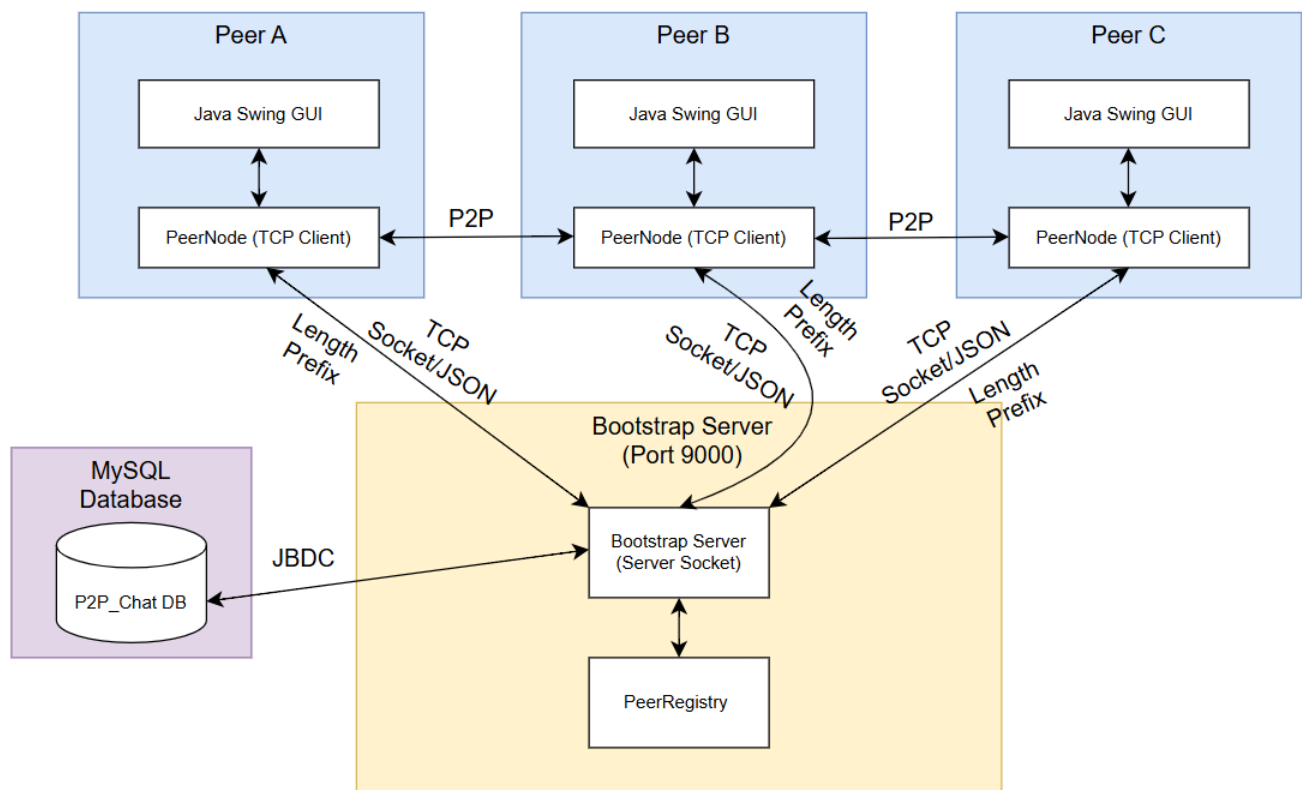
FlatLaf Dark Theme: Thay thế giao diện Swing cổ điển của hệ điều hành bằng bộ thư

viện FlatLaf với tông màu tối cao cấp Dark Mode lấy cảm hứng từ các ứng dụng chat hàng đầu hiện nay. FlatLaf hỗ trợ bo góc các nút nhấn, làm mịn phông chữ hiển thị và tùy biến thanh cuộn thông minh.

Giao diện Responsive và Thiết kế trực quan:

- Màn hình chat chia bố cục sidebar trái và khung chat phải rõ ràng.
- Tự động tính toán hiển thị bong bóng chat căn lề phải cho người gửi và lề trái cho người nhận kèm màu sắc tương phản nổi bật.
- Hỗ trợ tính năng hiển thị số lượng tin nhắn chưa đọc dưới dạng thẻ đỏ và các chỉ báo đang soạn tin nhắn sinh động.
- Hộp thoại Đăng nhập được thiết kế không viền hệ điều hành, bo cong các góc mềm mại và tích hợp mã nguồn kéo thả cửa sổ tự do để mang lại cảm giác cao cấp nhất cho người sử dụng.

2.3 Thiết kế kiến trúc hệ thống



Hình 2.5 Sơ đồ kiến trúc hệ thống

2.4 Thiết kế cơ sở dữ liệu

Các bảng CSDL

Bảng users (Lưu trữ thông tin tài khoản người dùng)

Tên thuộc tính	Kiểu dữ liệu	Chú thích
id	INT	Khóa chính, tự động tăng
username	VARCHAR(50)	Tên đăng nhập (Duy nhất, không được rỗng)
password_hash	VARCHAR(255)	Mật khẩu đã được băm an toàn
display_name	VARCHAR(100)	Tên hiển thị của người dùng
status	ENUM	Trạng thái hoạt động ('ONLINE', 'OFFLINE', 'AWAY', 'BUSY')
last_seen	TIMESTAMP	Lần cuối cùng hoạt động trực tuyến
created_at	TIMESTAMP	Ngày giờ tạo tài khoản
updated_at	TIMESTAMP	Ngày giờ cập nhật thông tin tài khoản gần nhất

Bảng messages (Lưu trữ lịch sử tất cả tin nhắn trực tiếp và tin nhắn nhóm)

Tên thuộc tính	Kiểu dữ liệu	Chú thích
id	BIGINT	Khóa chính, tự động tăng
message_id	VARCHAR(36)	ID định danh tin nhắn duy nhất (mã UUID)
from_user	VARCHAR(50)	Tên tài khoản người gửi tin nhắn
to_user	VARCHAR(50)	Tên tài khoản người nhận hoặc ID nhóm chat nhận
content	TEXT	Nội dung chi tiết của tin nhắn (có thể đã được mã hóa)
message_type	ENUM	Loại tin nhắn ('TEXT', 'FILE', 'IMAGE', 'SYSTEM', 'ENCRYPTED')

is_group_message	BOOLEAN	Xác định có phải là tin nhắn gửi vào nhóm không
group_id	INT	ID của nhóm nhận tin nhắn (nếu có)
is_read	BOOLEAN	Xác nhận tin nhắn đã được người nhận đọc hay chưa
is_delivered	BOOLEAN	Xác nhận tin nhắn đã chuyển tiếp thành công đến thiết bị nhận
is_encrypted	BOOLEAN	Xác định tin nhắn này có áp dụng mã hóa đầu-cuối không

Bảng chat_groups (Quản lý các nhóm chat trong hệ thống)

Tên thuộc tính	Kiểu dữ liệu	Chú thích
id	INT	Khóa chính, tự động tăng
group_id	VARCHAR(36)	ID định danh duy nhất của nhóm (mã UUID)
group_name	VARCHAR(100)	Tên hiển thị của nhóm chat
description	TEXT	Mô tả ngắn gọn về nhóm chat
created_by	VARCHAR(50)	Tên tài khoản người khởi tạo nhóm (Admin nhóm)
max_members	INT	Số lượng thành viên tham gia tối đa cho phép
created_at	TIMESTAMP	Ngày giờ khởi tạo nhóm
updated_at	TIMESTAMP	Ngày giờ cập nhật thông tin nhóm gần nhất

Bảng group_members (Quản lý danh sách thành viên tham gia các nhóm)

Tên thuộc tính	Kiểu dữ liệu	Chú thích
id	INT	Khóa chính, tự động tăng
group_id	VARCHAR(36)	ID nhóm chat tham gia
username	VARCHAR(50)	Tên tài khoản thành viên trong nhóm
role	ENUM	Quyền hạn của thành viên ('ADMIN', 'MEMBER')
joined_at	TIMESTAMP	Ngày giờ thành viên gia nhập nhóm chat
id	INT	Khóa chính, tự động tăng
group_id	VARCHAR(36)	ID nhóm chat tham gia
username	VARCHAR(50)	Tên tài khoản thành viên trong nhóm

Bảng offline_messages (Lưu trữ tin nhắn tạm thời chờ gửi khi người nhận online)

Tên thuộc tính	Kiểu dữ liệu	Chú thích
id	BIGINT	Khóa chính, tự động tăng
message_id	VARCHAR(36)	ID định danh của tin nhắn gốc
from_user	VARCHAR(50)	Tên tài khoản người gửi tin nhắn
to_user	VARCHAR(50)	Tên tài khoản người nhận đang ngoại tuyến (Offline)
content	TEXT	Nội dung tin nhắn đang được lưu trữ tạm thời
message_type	VARCHAR(20)	Định dạng loại tin nhắn gửi đi
is_group_message	BOOLEAN	Xác định xem tin nhắn ngoại tuyến thuộc nhóm hay không
group_id	VARCHAR(36)	ID của nhóm chat chứa tin nhắn tạm thời

Bảng peer_registry (Đăng ký địa chỉ mạng IP/Port của các peer đang online)

Tên thuộc tính	Kiểu dữ liệu	Chú thích
id	INT	Khóa chính, tự động tăng
username	VARCHAR(50)	Tên tài khoản định danh duy nhất của Peer
ip_address	VARCHAR(45)	Địa chỉ IP của Client hiện tại đang kết nối
tcp_port	INT	Cổng kết nối TCP do Client đăng ký lắng nghe
is_online	BOOLEAN	Xác nhận Peer đang ở trạng thái trực tuyến hoạt động
last_heartbeat	TIMESTAMP	Lần cuối cùng hệ thống nhận được tín hiệu giữ kết nối (Heartbeat)
registered_at	TIMESTAMP	Ngày giờ Peer thực hiện kết nối đăng ký lên Server

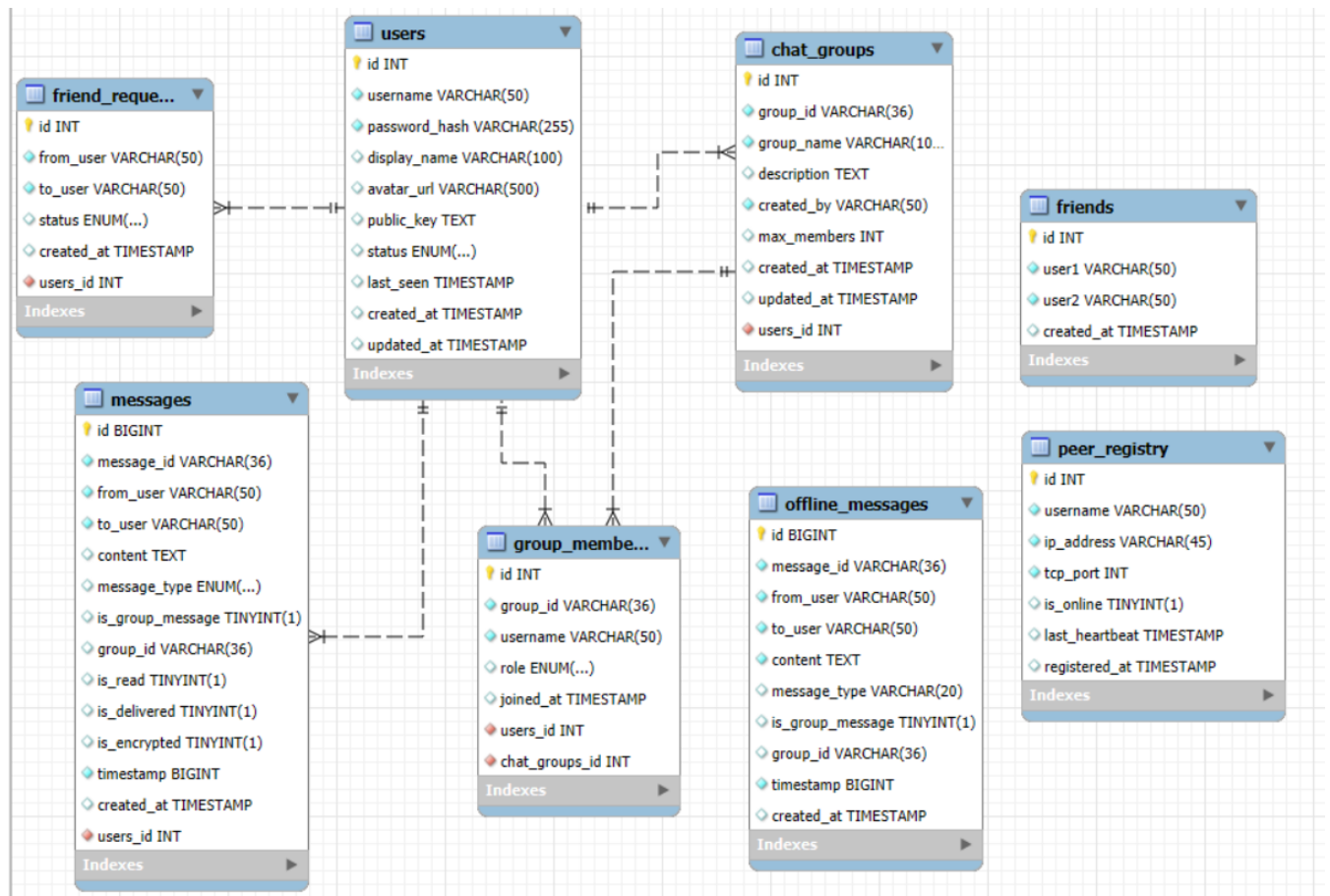
Bảng friends (Lưu thông tin quan hệ bạn bè đã thiết lập giữa các tài khoản)

Tên thuộc tính	Kiểu dữ liệu	Chú thích
id	INT	Khóa chính, tự động tăng
user1	VARCHAR(50)	Tên tài khoản của người dùng thứ nhất trong mối quan hệ
user2	VARCHAR(50)	Tên tài khoản của người dùng thứ hai trong mối quan hệ
created_at	TIMESTAMP	Ngày giờ thiết lập mối quan hệ bạn bè giữa cả hai

Bảng friend_requests (Lưu lịch sử yêu cầu kết bạn)

Tên thuộc tính	Kiểu dữ liệu	Chú thích
id	INT	Khóa chính, tự động tăng
from_user	VARCHAR(50)	Tên tài khoản của người gửi lời mời kết bạn
to_user	VARCHAR(50)	Tên tài khoản của người nhận lời mời kết bạn
status	ENUM	Trạng thái yêu cầu kết bạn ('PENDING', 'ACCEPTED', 'REJECTED')
created_at	TIMESTAMP	Ngày giờ tạo lời mời yêu cầu kết bạn

Sơ đồ liên kết cơ sở dữ liệu



Hình 2.6 Sơ đồ liên kết cơ sở dữ liệu

CHƯƠNG 3: THỰC NGHIỆM, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

3.1 Môi trường triển khai và công nghệ sử dụng

Ngôn ngữ: Java (phiên bản Java 17+).

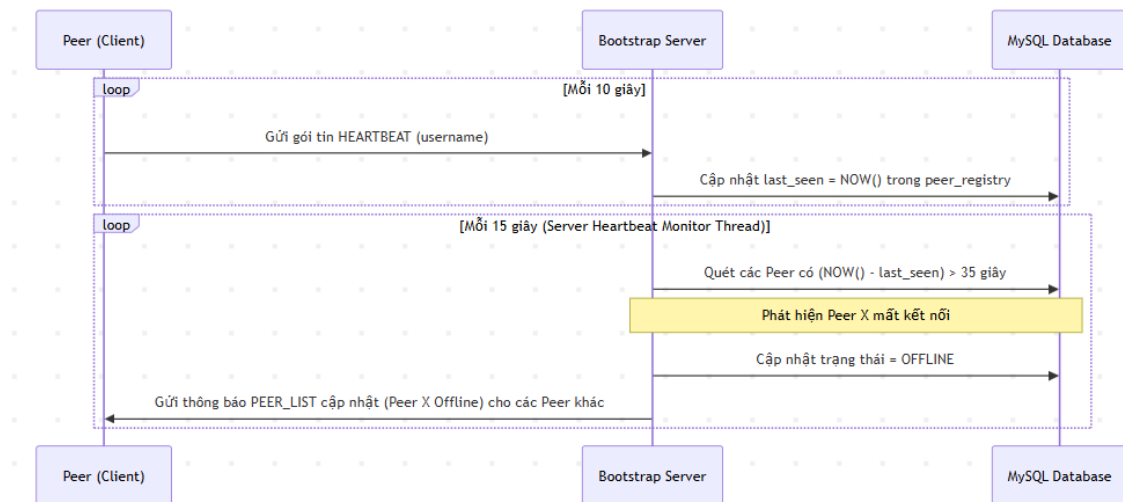
Hệ quản trị CSDL: MySQL 8.0.

Thư viện hỗ trợ: Maven, Gson (xử lý JSON), FlatLaf (giao diện tối cao cấp), HikariCP (Connection Pool tối ưu DB).

3.2 Các cơ chế cốt lõi của hệ thống

3.2.1 Cơ chế Peer Discovery và quản lý trạng thái (HeartBeat)

Để duy trì danh sách mạng ngang hàng hoạt động thời gian thực mà không tiêu tốn quá nhiều tài nguyên mạng, hệ thống áp dụng cơ chế quản lý trạng thái thông qua các gói tin Heartbeat định kỳ.



Hình 3.1 Cơ chế quản lý trạng thái

Lưu trữ thông tin Peer (Peer Registry): Khi một Peer đăng nhập thành công vào hệ thống, thông tin kết nối bao gồm: username, IP Address, Port và thời gian tương tác cuối cùng last_seen sẽ được lưu và cập nhật trong bảng peer_registry ở cơ sở dữ liệu MySQL tập trung của Bootstrap Server. Nhờ bảng này, Bootstrap Server đóng vai trò như một thư mục chỉ dẫn để các Peer tìm kiếm thông tin IP/Port của nhau.

Cơ chế Quét Heartbeat:

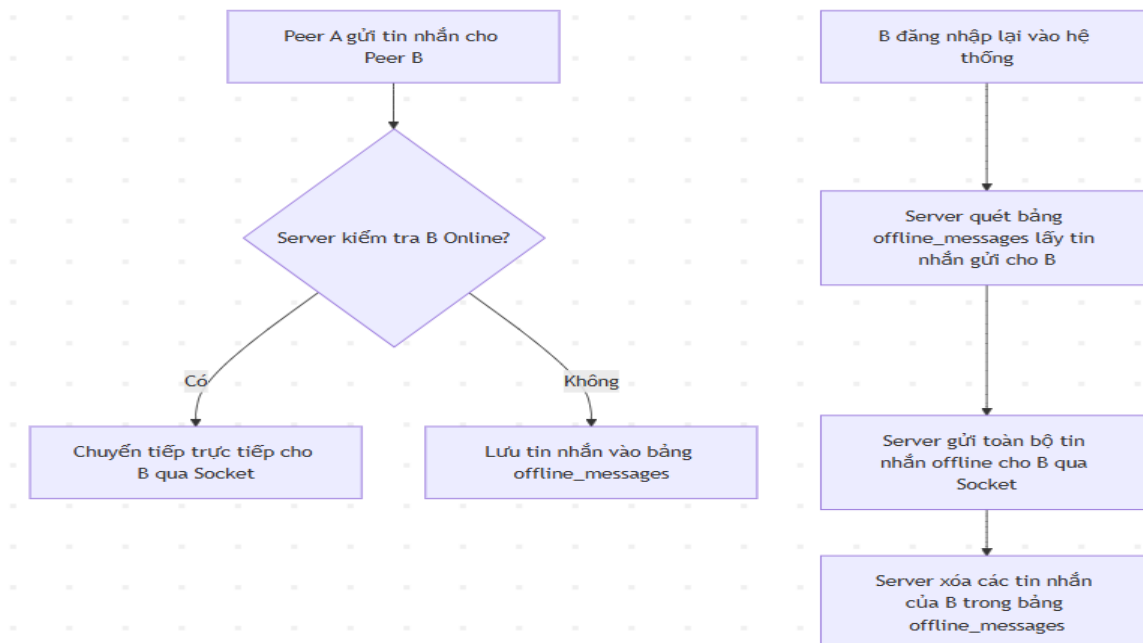
Tại phía Client: Sau khi đăng nhập, Client khởi chạy một luồng ngầm định kỳ gửi gói tin có kiểu HEARTBEAT kèm theo username của mình lên Server (mỗi 10 giây một lần).

Tại phía Server: Server duy trì một tiến trình nền (Heartbeat Monitor Thread) chạy định kỳ 15 giây một lần. Tiến trình này sẽ thực hiện quét bảng peer_registry trong cơ sở dữ liệu để tìm ra các Peer có khoảng thời gian không gửi tín hiệu tương tác vượt quá 35 giây (last_seen cách hiện tại > 35s).

Xử lý ngắt kết nối: Khi phát hiện một Peer bị timeout (do mất mạng đột ngột hoặc tắt ứng dụng không qua thủ tục logout chuẩn), Server tự động chuyển trạng thái của Peer đó sang OFFLINE trong database, giải phóng kết nối Socket, đồng thời gửi một gói tin cập nhật danh sách Peer (PEER_LIST) mới tới toàn bộ các Peer khác đang online để cập nhật giao diện thời gian thực.

3.2.2 Cơ chế Tin nhắn ngoại tuyến

Hệ thống hỗ trợ gửi tin nhắn ngay cả khi người nhận không trực tuyến thông qua thuật toán Lưu trữ và Chuyển tiếp (Store-and-Forward) trung gian qua Server.



Hình 3.2 Cơ chế lưu trữ và chuyển tiếp

Thuật toán gửi tin nhắn:

1. Khi Peer A muốn gửi tin nhắn tới Peer B, Client sẽ gửi gói tin qua Server.
2. Server tiếp nhận gói tin và kiểm tra trạng thái hoạt động của Peer B trong bộ nhớ đệm (hoặc bảng peer_registry).
3. Trường hợp Peer B Online: Server chuyển tiếp trực tiếp gói tin nhắn qua luồng Socket đang mở của Peer B.
4. Trường hợp Peer B Offline: Server chuyển hướng gói tin nhắn đó vào bảng lưu trữ tạm thời offline_messages trong database (chứa các thông tin: người gửi, người nhận, nội dung mã hóa, thời gian gửi).

Đồng bộ khi đăng nhập lại: Khi Peer B đăng nhập lại vào hệ thống, sau khi hoàn tất xác thực, Server sẽ kích hoạt một trình quét để kiểm tra bảng offline_messages. Tất cả các tin nhắn gửi đến B đang lưu trữ tạm thời sẽ được truy vấn, sắp xếp theo thời gian, đóng gói và gửi về cho B thông qua kết nối Socket mới thiết lập. Sau khi nhận được xác nhận gửi thành công, Server sẽ xóa các bản ghi tạm thời này trong database để giải phóng bộ nhớ.

3.2.3 Cơ chế đồng bộ trạng thái gõ chữ và đã đọc

Để tối ưu hóa trải nghiệm người dùng tương tự các ứng dụng chat hiện đại, hệ thống triển khai bộ đồng bộ trạng thái trực quan



Hình 3.3 Cơ chế đồng bộ trạng thái gõ chữ và đã đọc

Đồng bộ trạng thái gõ chữ: Tại khung nhập liệu của Client, hệ thống lắng nghe sự kiện nhấn phím từ người dùng. Khi phát hiện người dùng bắt đầu nhập văn bản, ứng dụng sẽ gửi gói tin có kiểu TYPING với payload `isTyping = true` tới đối phương. Ngược lại, khi người dùng không nhập liệu trong một khoảng thời gian ngắn hoặc sau khi nhấn nút gửi, một sự kiện TYPING với payload `isTyping = false` được truyền đi. Phía đối phương khi nhận gói tin này sẽ hiển thị hoặc ẩn dòng chữ động báo trạng thái gõ chữ ở đầu cửa sổ chat.

Đồng bộ trạng thái đã gửi, đã nhận và đã đọc:

Trạng thái Đã gửi: Nhãn hiển thị ngay khi người dùng nhấn nút Gửi tin nhắn thành công lên hệ thống mạng.

Trạng thái Đã nhận: Ngay khi Client của người nhận nhận được gói tin từ Socket và hiển thị lên giao diện, nó tự động gửi phản hồi xác nhận ngược lại cho người gửi để đổi nhãn thành "Đã nhận".

Trạng thái Đã đọc: Khi người nhận thực sự mở cửa sổ chat của người gửi đó (hoặc nếu đang mở sẵn cửa sổ chat đó khi tin nhắn mới đến), Client người nhận sẽ gửi gói tin READ_RECEIPT kèm theo ID của tin nhắn. Người gửi khi nhận được gói tin này sẽ thay đổi trạng thái nhãn tin nhắn thành "Đã đọc" và đổi màu nhãn sang màu xanh lá. Trạng thái này cũng được đồng bộ và cập nhật cập nhật vào bảng cơ sở dữ liệu messages.

3.2.4 Cơ chế Trích dẫn trả lời & Chuyển tiếp tin nhắn

Để nâng cao khả năng tương tác, hệ thống hỗ trợ cơ chế phản hồi theo ngữ cảnh và luân chuyển tin nhắn:

Cơ chế Trả lời: Mỗi tin nhắn hiển thị trên giao diện đều được liên kết với một ID duy nhất (messageId). Khi người dùng chọn "Trả lời" từ menu ngữ cảnh của một tin nhắn, hệ thống sẽ lưu thông tin replyToId, replyToSender (username thực tế của người gửi gốc) và replyToContent. Khi tin nhắn mới được gửi đi, các thông tin này được đính kèm vào phần cấu trúc JSON payload của gói tin. Phía nhận tin sẽ trích xuất thông tin này để dựng một khung trích dẫn nhỏ nằm ngay phía trên tin nhắn trả lời giúp cuộc trò chuyện mạch lạc.

Cơ chế Chuyển tiếp: Người dùng có thể sao chép nhanh nội dung của một tin nhắn và chuyển tiếp tới một người dùng khác hoặc một nhóm chat khác. Khi chuyển tiếp, hệ thống tự động gán thuộc tính isForwarded = true trong payload tin nhắn giúp phân biệt tin nhắn soạn thảo trực tiếp với tin nhắn được chia sẻ lại từ nguồn khác.

3.2.5 Cơ chế Bảo mật & Mã hóa gói tin

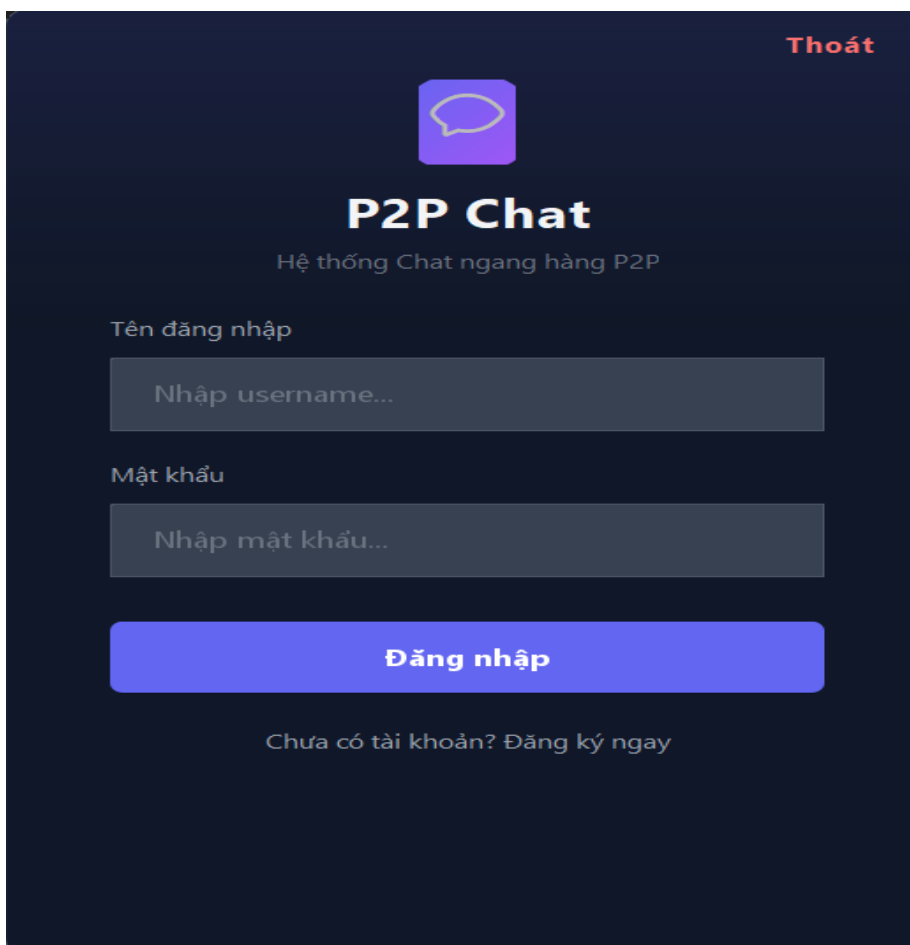
Đảm bảo an toàn thông tin đầu cuối End-to-End Encryption cho các cuộc trò chuyện riêng tư:

Mã hóa hỗn hợp: Hệ thống sử dụng sự kết hợp giữa mã hóa bất đối xứng RSA (2048-bit) để trao đổi khóa và mã hóa đối xứng AES-GCM (256-bit) để mã hóa nội dung tin nhắn thực tế.

Quy trình trao đổi: Khi hai Peer thiết lập kết nối trực tiếp, chúng sẽ gửi khóa công khai RSA cho nhau. Khi gửi tin nhắn, Client sẽ tự động tạo ngẫu nhiên một khóa AES dùng một lần, mã hóa nội dung tin nhắn bằng khóa AES này, sau đó mã hóa chính khóa AES đó bằng khóa công khai RSA của người nhận rồi gửi đi. Phía nhận sẽ dùng khóa bí mật RSA của mình để giải mã ra khóa AES, sau đó dùng khóa AES để giải mã nội dung tin nhắn. Cơ chế này đảm bảo ngay cả khi tin nhắn bị chặn trên đường truyền hoặc cơ sở dữ liệu trung tâm bị rò rỉ, nội dung tin nhắn vẫn hoàn toàn được bảo mật.

3.3 Giao diện hệ thống

Giao diện đăng nhập hệ thống: Người dùng nhập tên và mật khẩu với tài khoản đã đăng ký trước đây rồi click vào nút Đăng nhập để vào hệ thống



Thoát



P2P Chat

Hệ thống Chat ngang hàng P2P

Tên đăng nhập

Nhập username...

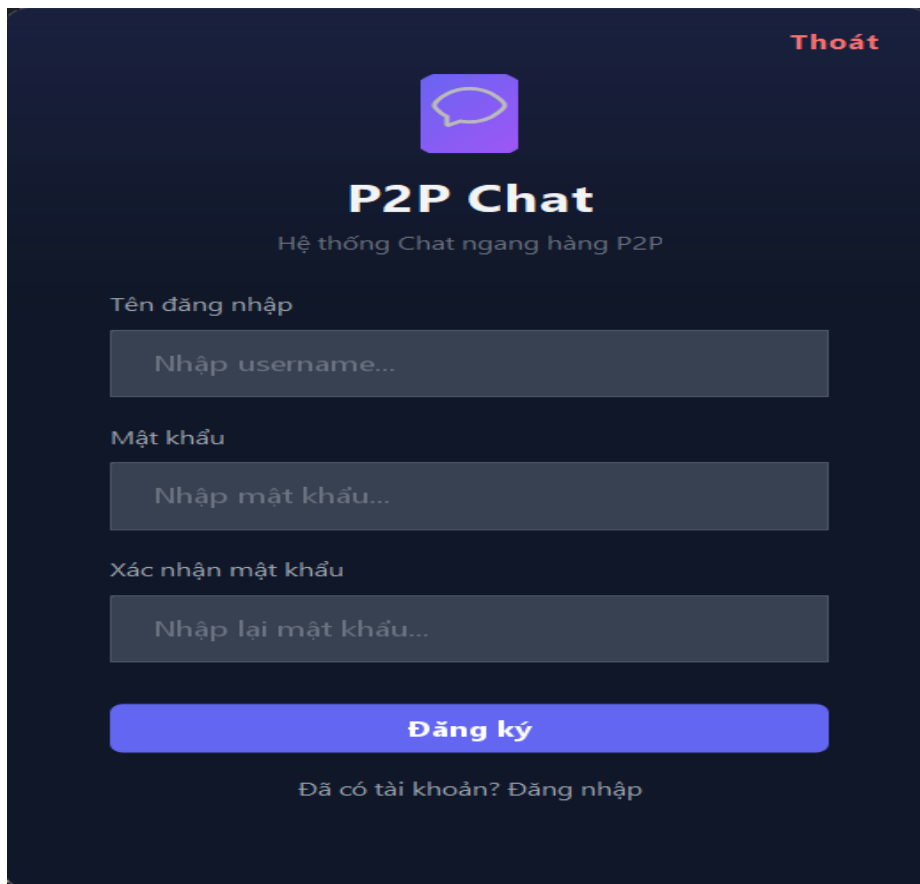
Mật khẩu

Nhập mật khẩu...

Đăng nhập

Chưa có tài khoản? Đăng ký ngay

Giao diện đăng ký: Người dùng thực hiện nhập tên đăng nhập, mật khẩu và xác nhận mật khẩu rồi nhấn vào nút đăng ký để đăng ký tài khoản



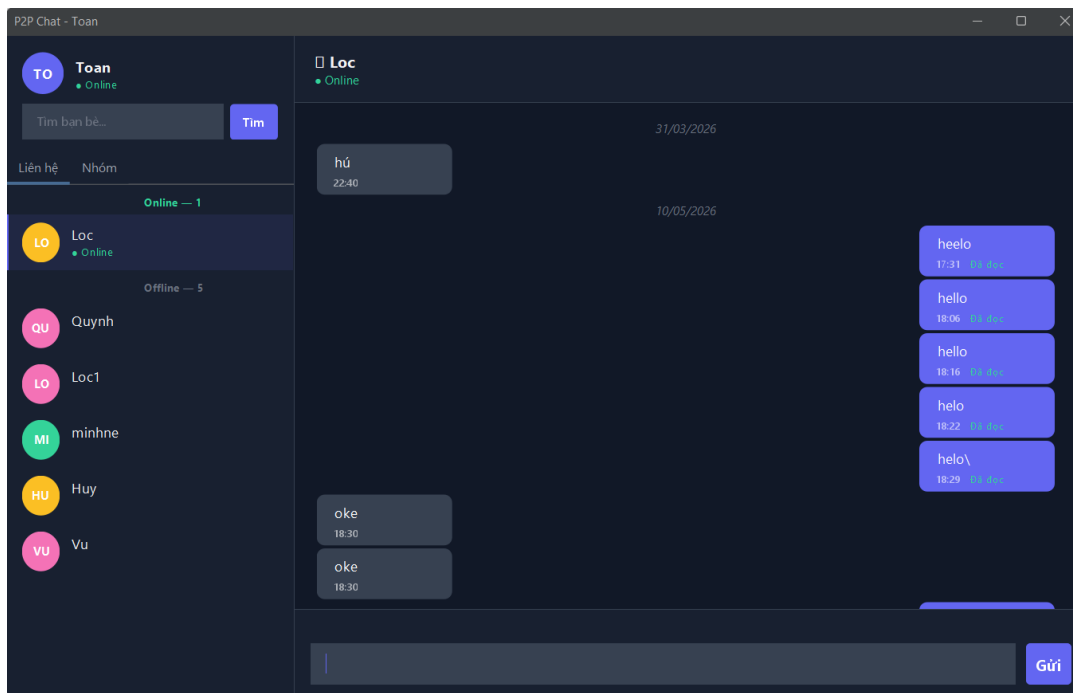
The registration form for P2P Chat is displayed on a dark blue background. At the top right is a red "Thoát" button. In the center is a purple speech bubble icon. Below it, the title "P2P Chat" is shown in large white letters, followed by the subtitle "Hệ thống Chat ngang hàng P2P" in smaller white letters. The form consists of four input fields: "Tên đăng nhập" (Username) with placeholder "Nhập username...", "Mật khẩu" (Password) with placeholder "Nhập mật khẩu...", and "Xác nhận mật khẩu" (Confirm password) with placeholder "Nhập lại mật khẩu...". Below these fields is a large blue button labeled "Đăng ký". At the bottom, there is a link "Đã có tài khoản? Đăng nhập" in white text.

Giao diện chính của hệ thống:

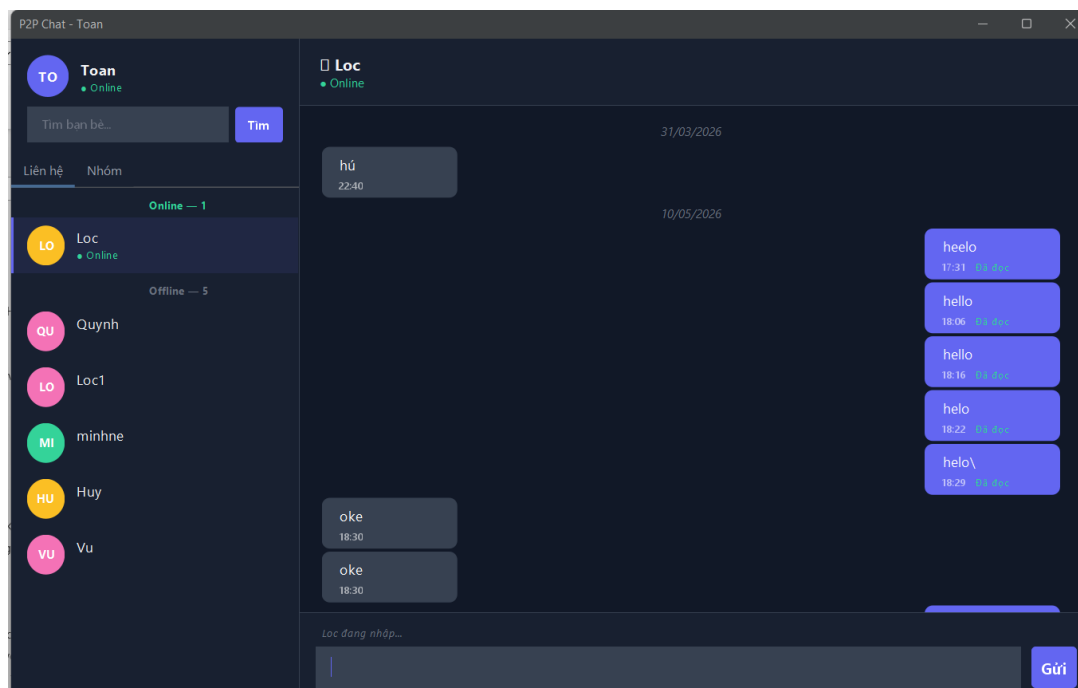


Giao diện khi vào một đoạn chat:

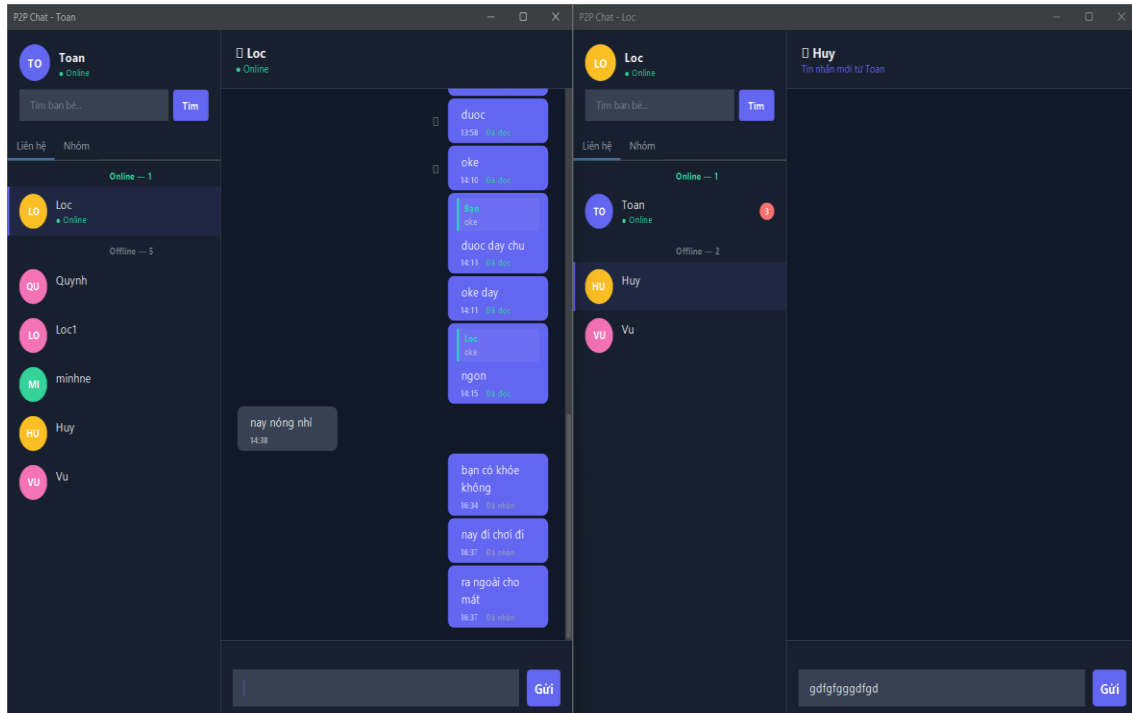
Giao diện hiển thị toàn bộ lịch sử tin nhắn của các ngày, Tên, trạng thái hoạt động và hộp thoại nhập tin để gửi tin nhắn



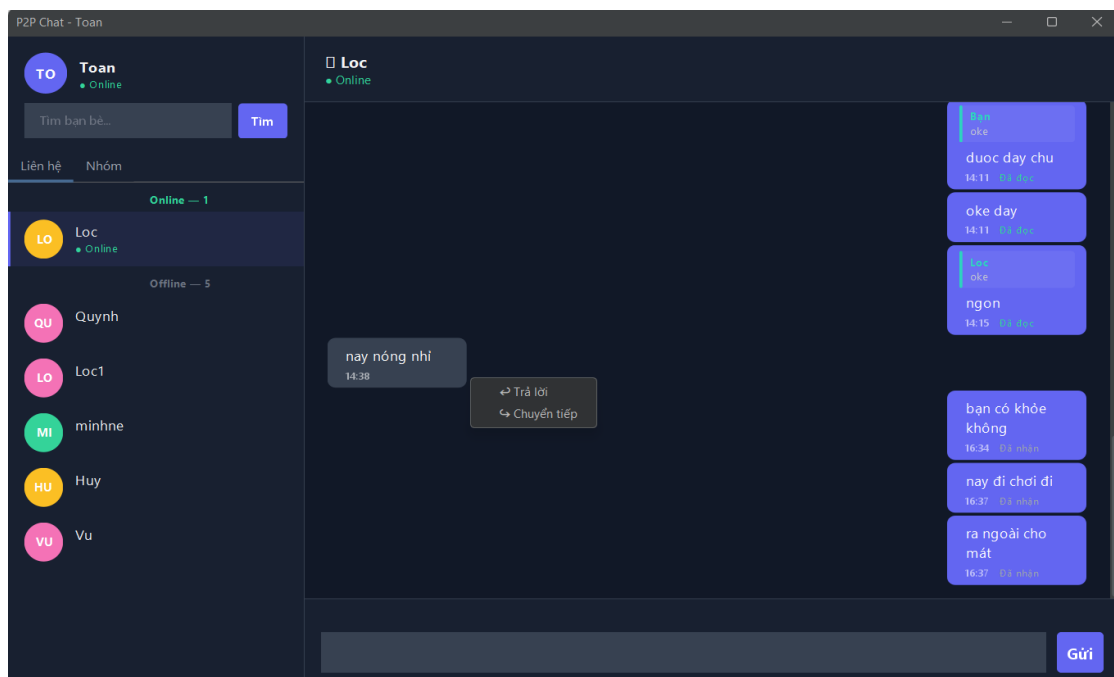
Khi đối phương nhập tin nhắn hệ thống hiển thị trạng thái đang nhập

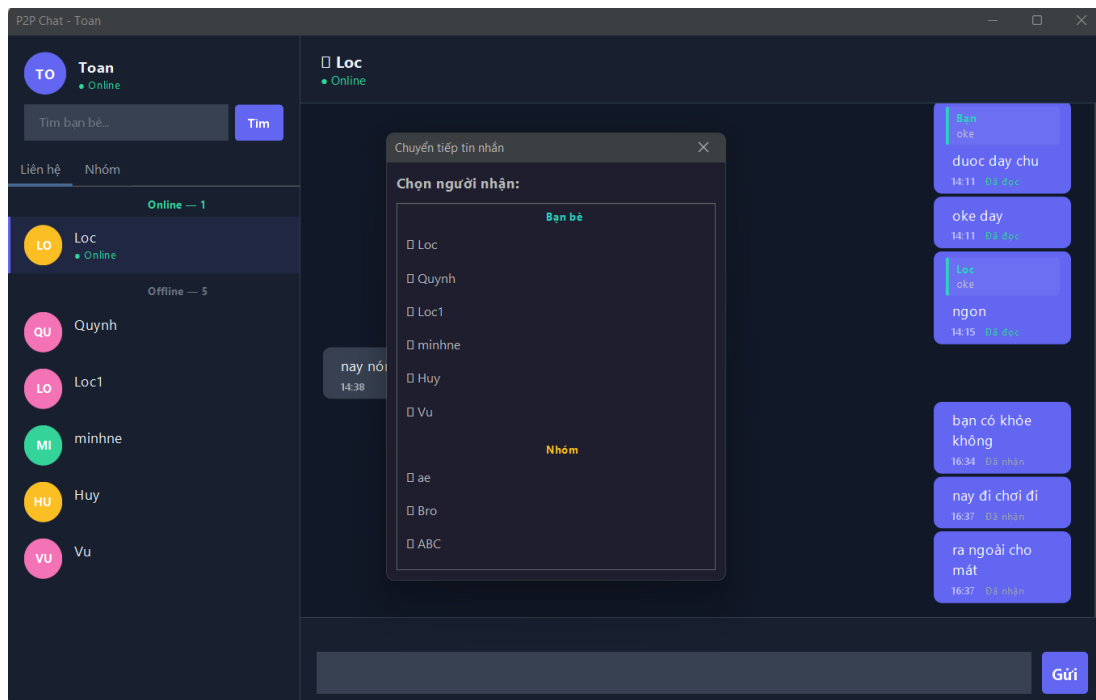


Trạng thái khi 1 người nhắn tin mà người kia chưa đọc tin nhắn: hiển thị số tin nhắn chưa đọc, trạng thái tin nhắn (Đã nhận); khi gửi tin nhắn thông báo tin nhắn sẽ hiển thị ở dưới tên người mình đang xem đoạn chat

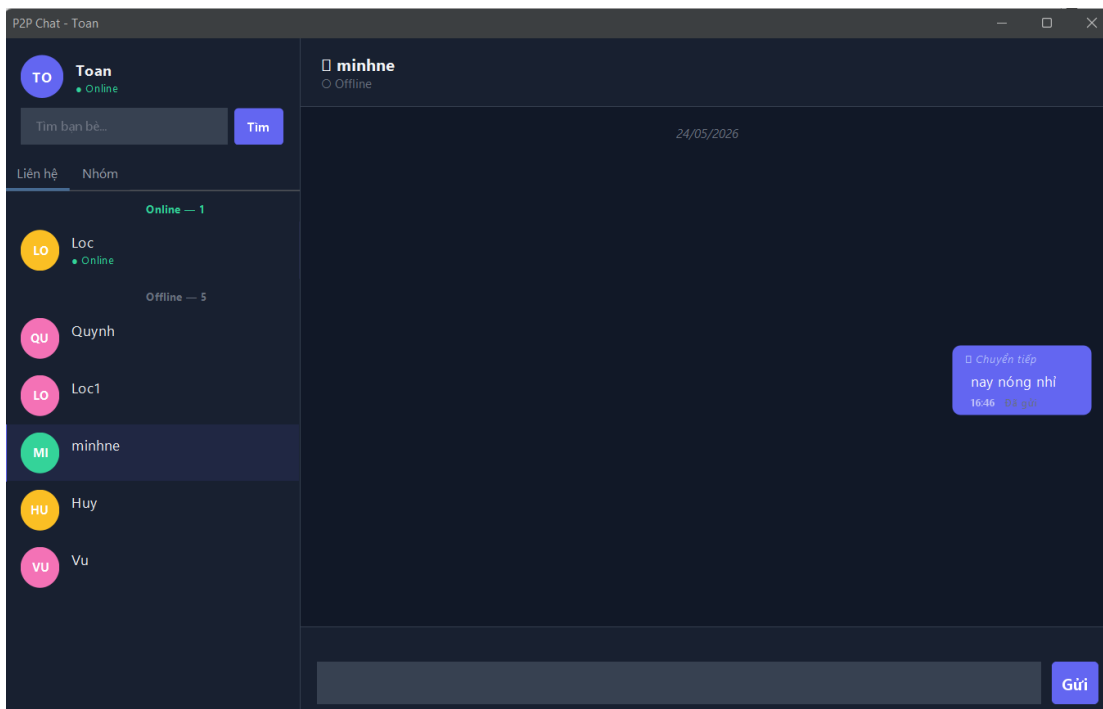


Người dùng có thể trả lời tin nhắn hoặc chuyển tiếp tin nhắn của mình hay của đối phương cho những người khác hoặc các nhóm

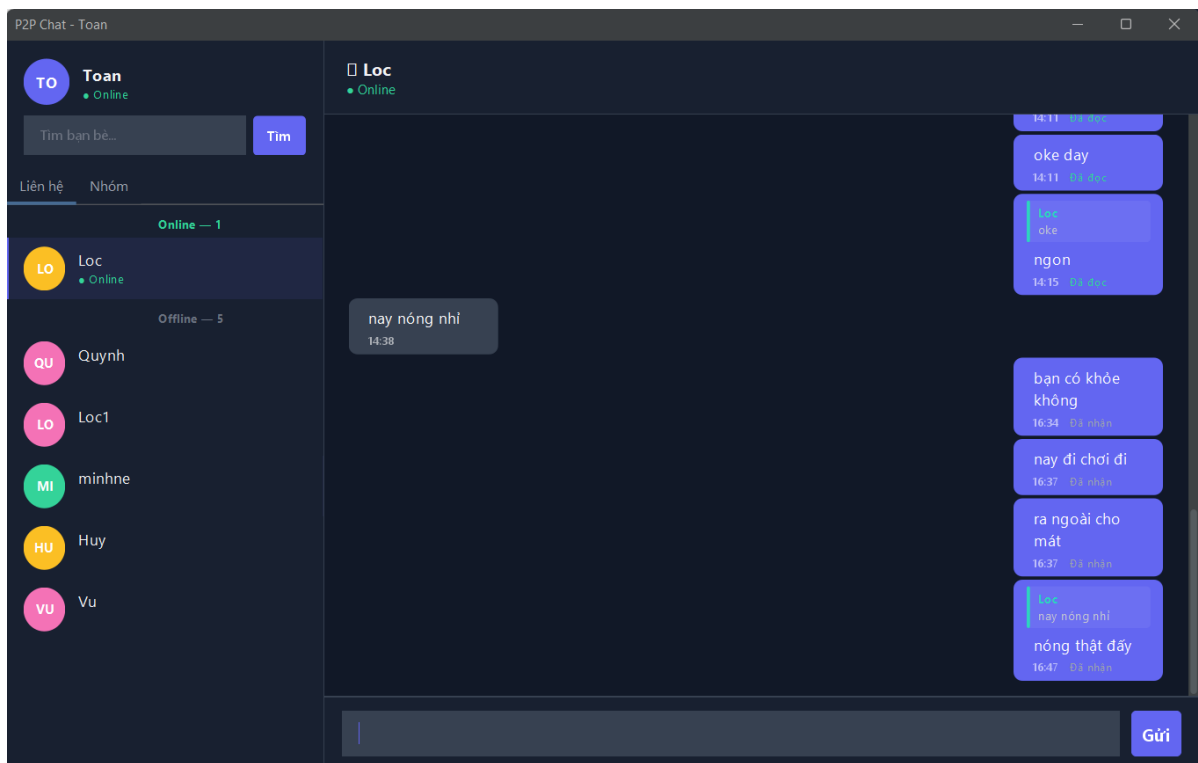




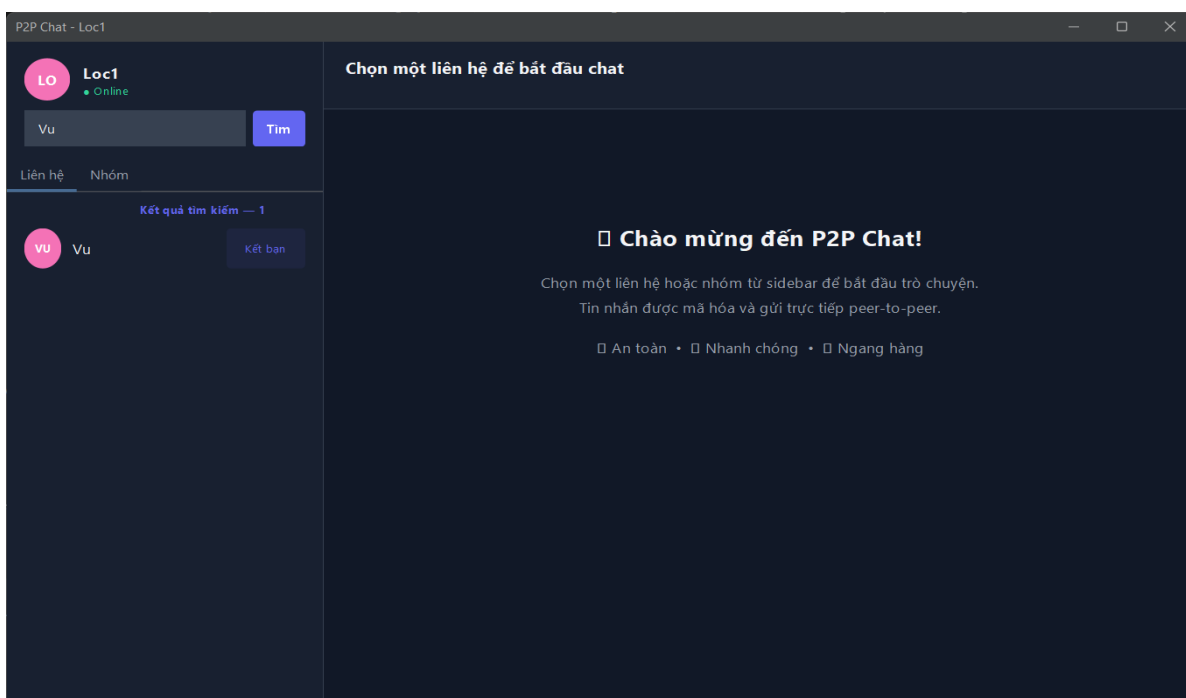
Chuyển tiếp tin nhắn cho minhne



Trả lời tin nhắn

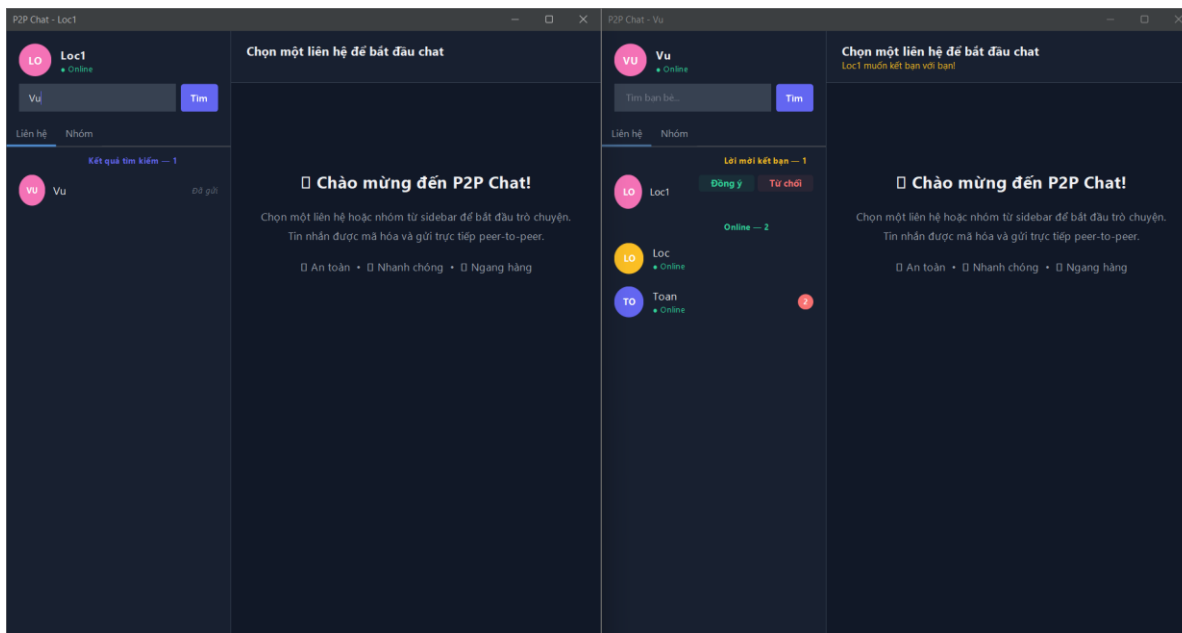


Giao diện tìm kiếm: Người dùng có thể tìm kiếm người khác thông qua nhập tên người khác trên thanh tìm kiếm rồi bấm nút tìm, danh sách người dùng muốn tìm sẽ hiện ra

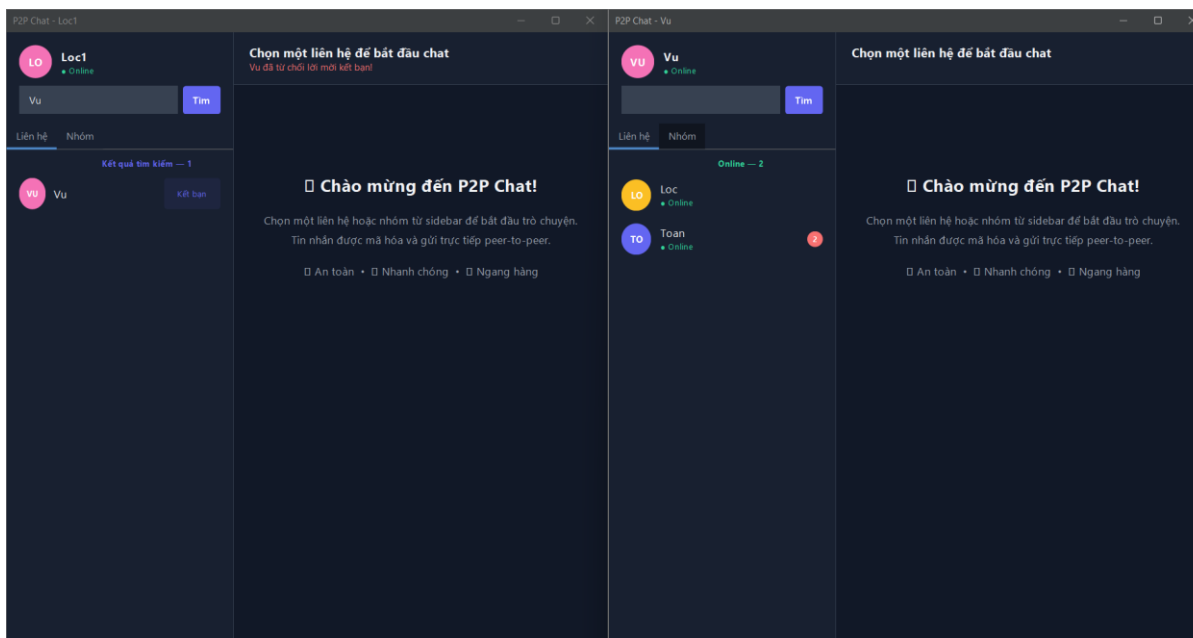


Giao diện khi gửi lời mời kết bạn:

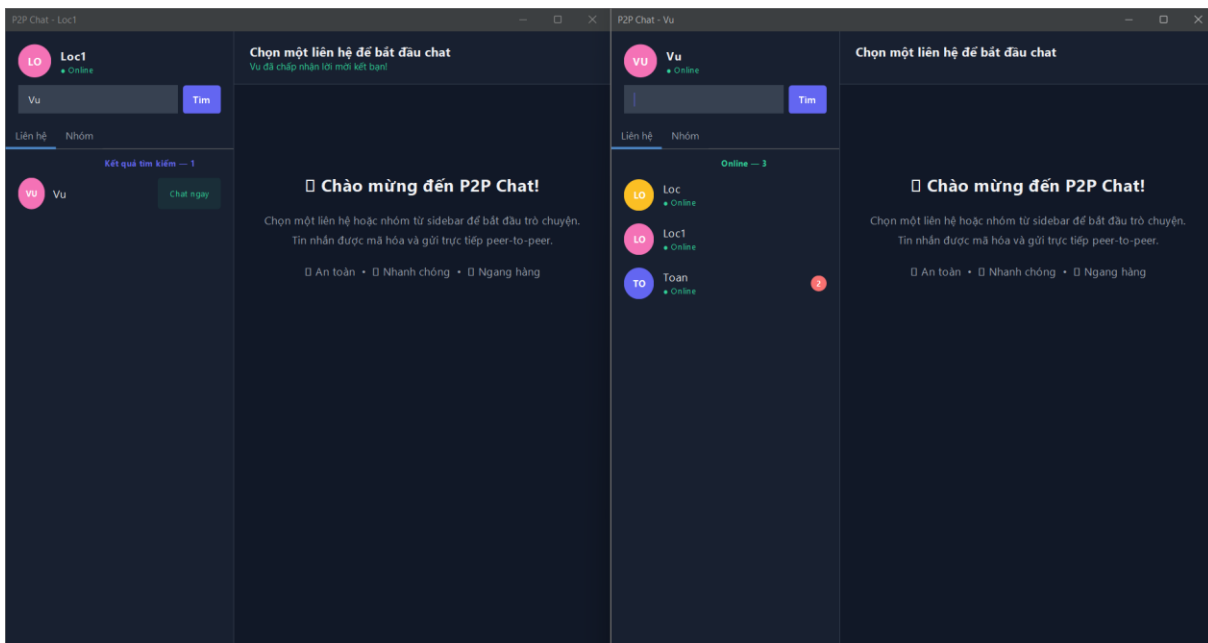
Hệ thống hiển thị thông báo lời mời kết bạn và hộp thoại “Lời mời kết bạn”



Khi từ chối kết bạn thông báo từ chối được hiển thị và hộp thoại “Lời mời kết bạn” biến mất

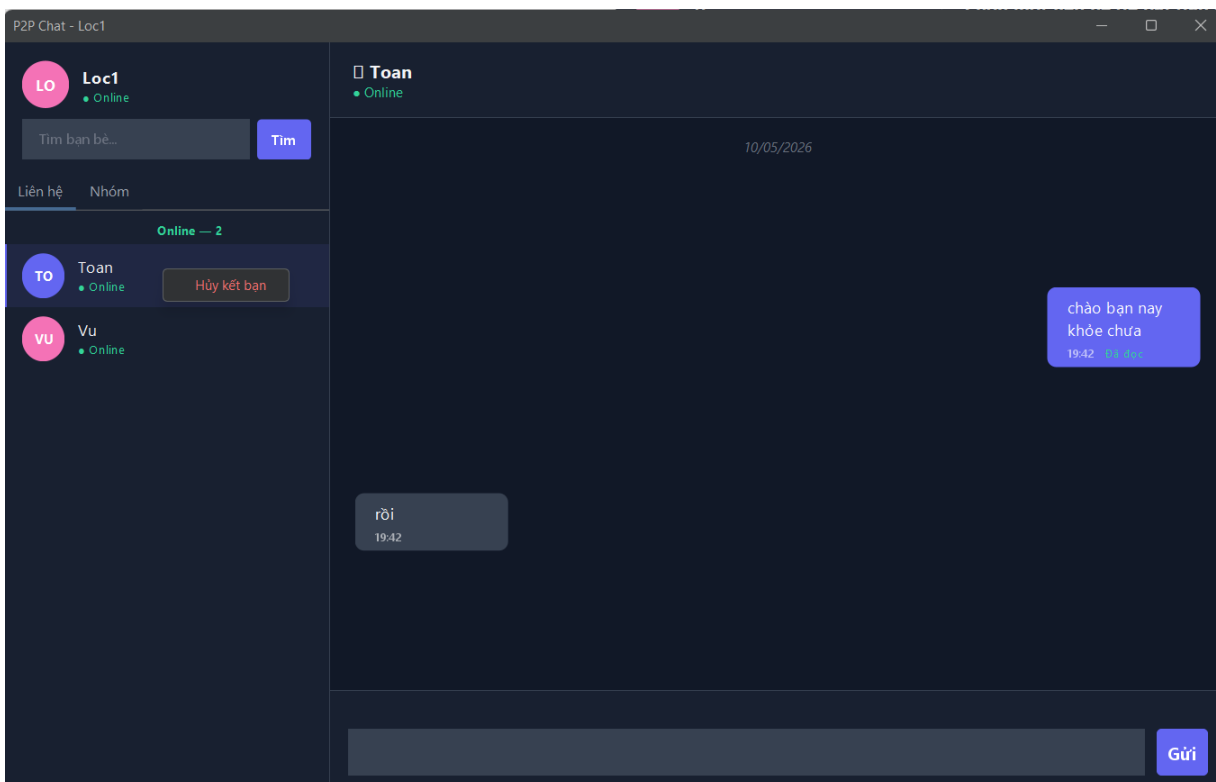


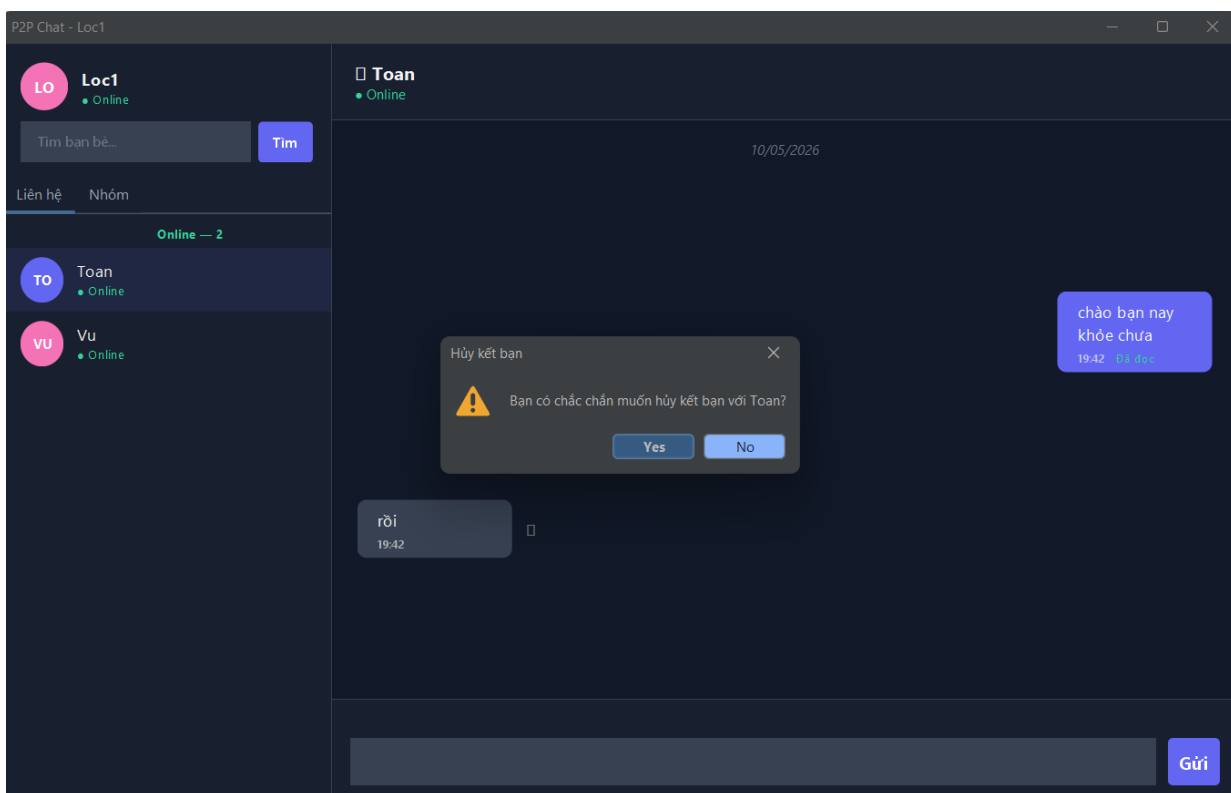
Khi đồng ý kết bạn hộp thoại tin nhắn với người vừa mới kết bạn sẽ hiện ra (Bên đồng ý) và chữ “Chat ngay” sẽ hiện ra (Người gửi)



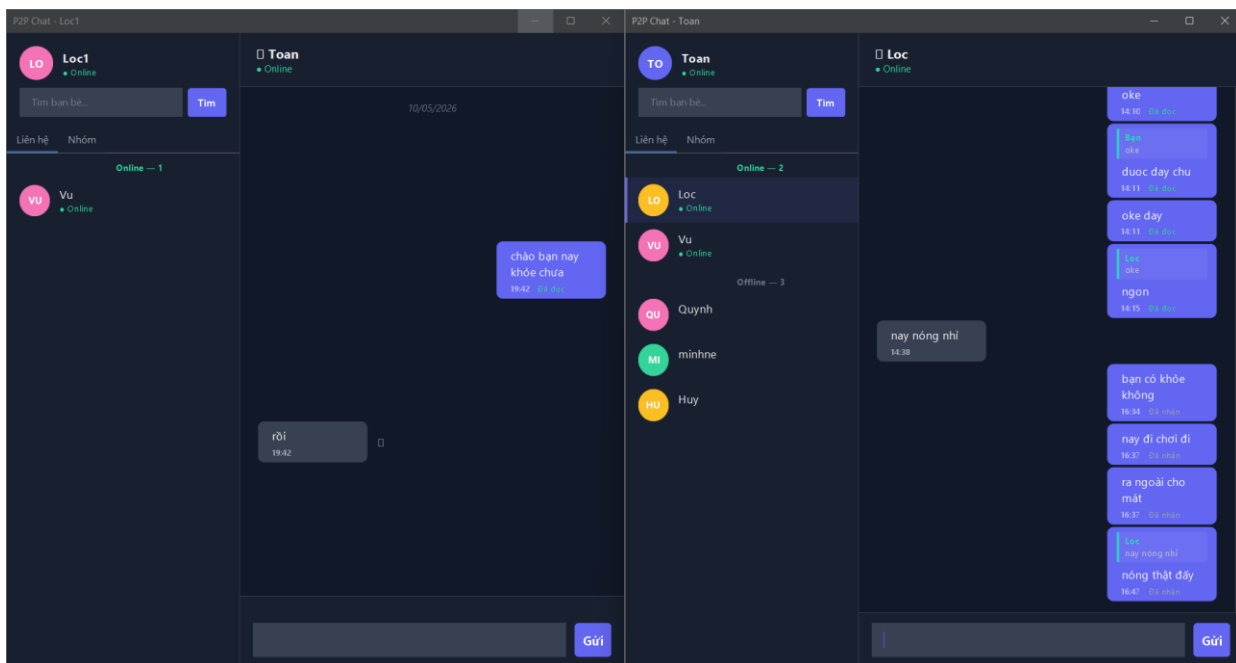
Giao diện hủy kết bạn:

Người dùng có thể hủy kết bạn với bất kỳ ai đã kết bạn và nhắn tin

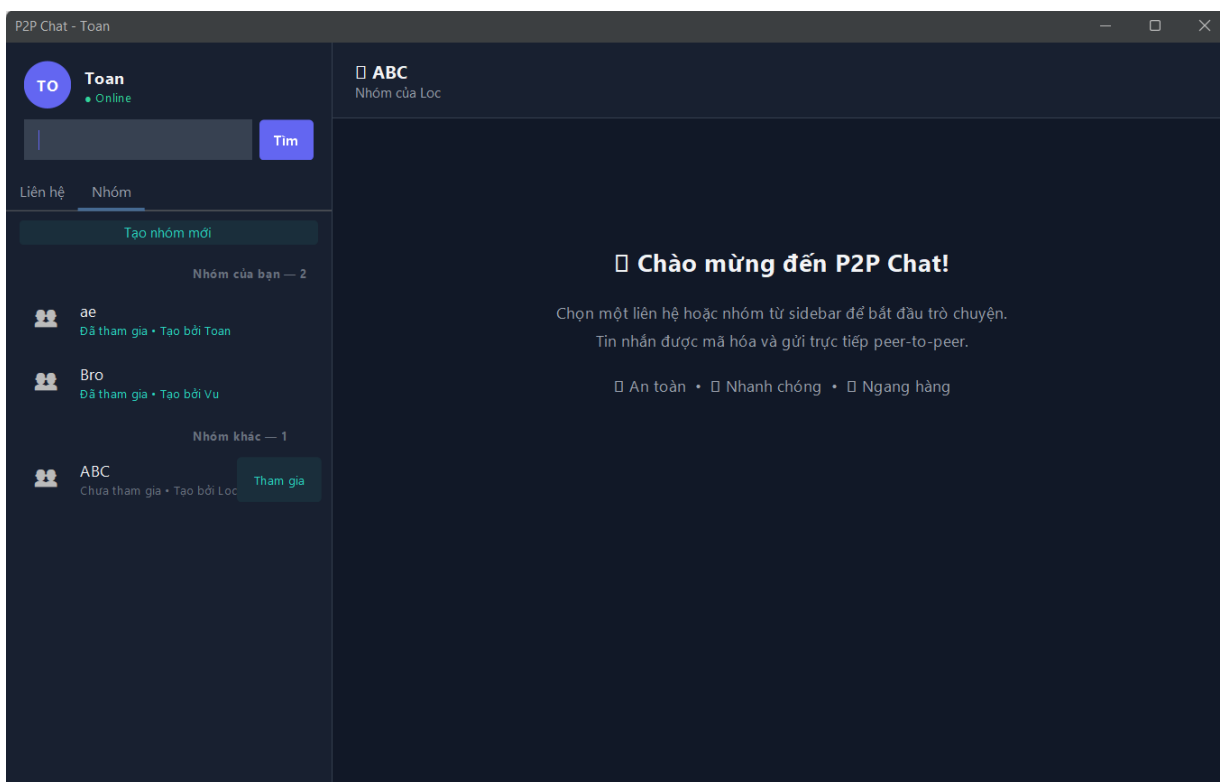




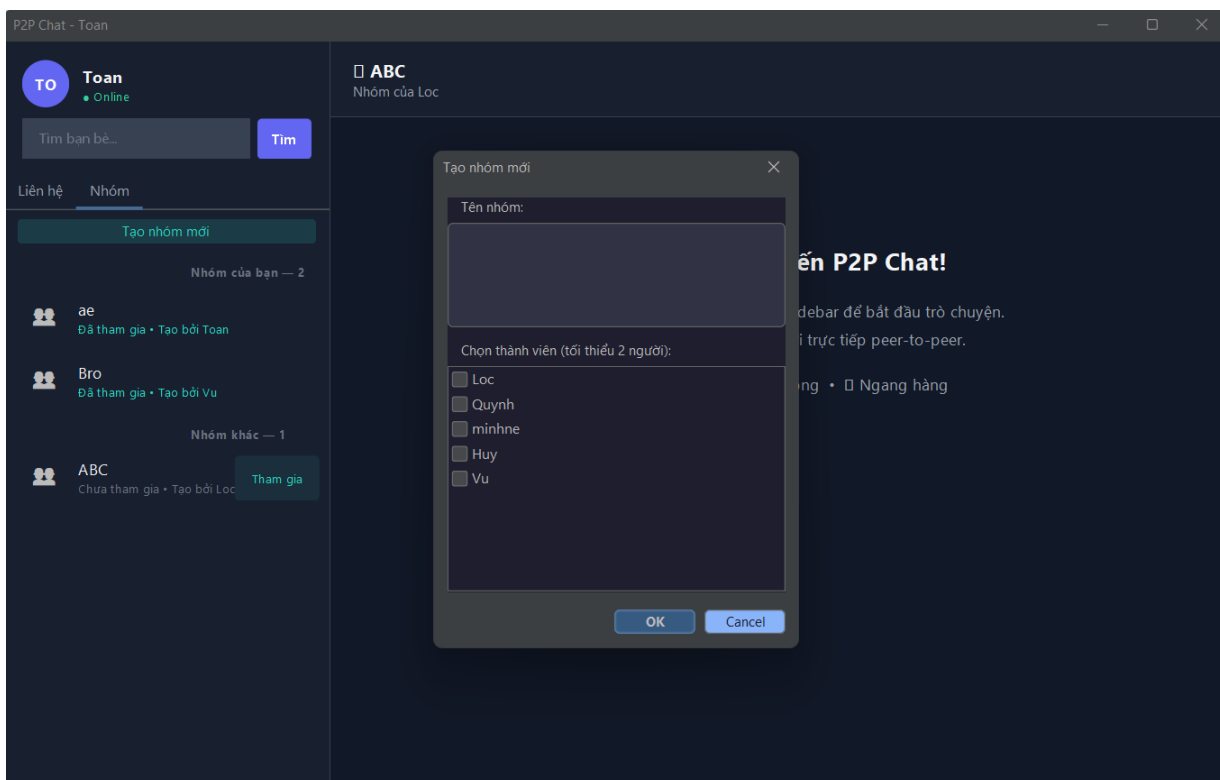
Hộp thoại tin nhắn của 2 người sẽ biến mất khi hủy kết bạn



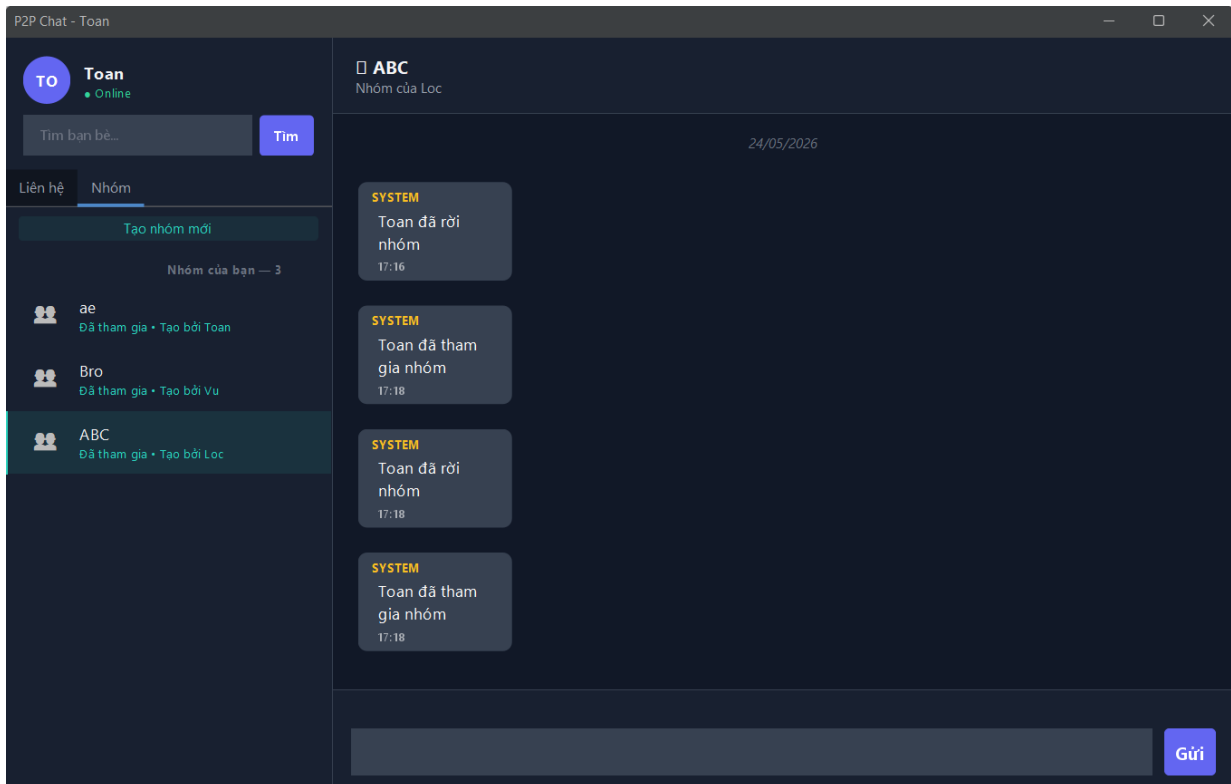
Giao diện chat nhóm



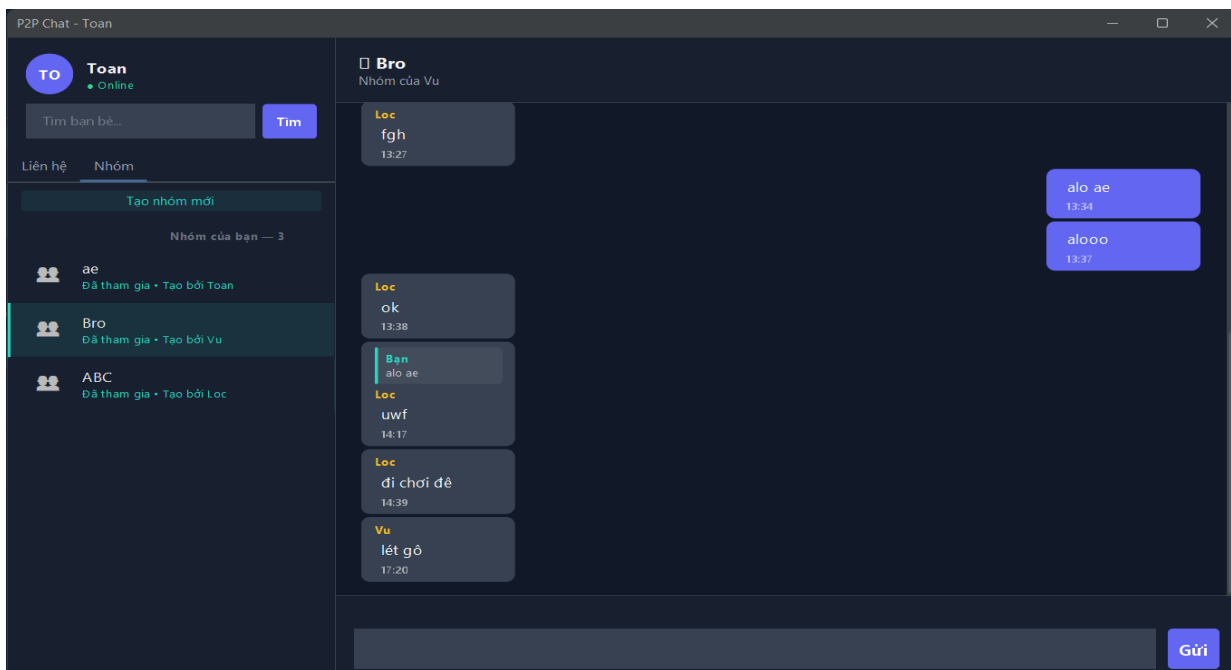
Người dùng có thể tạo nhóm mới với người dùng sẽ là Admin của nhóm đó



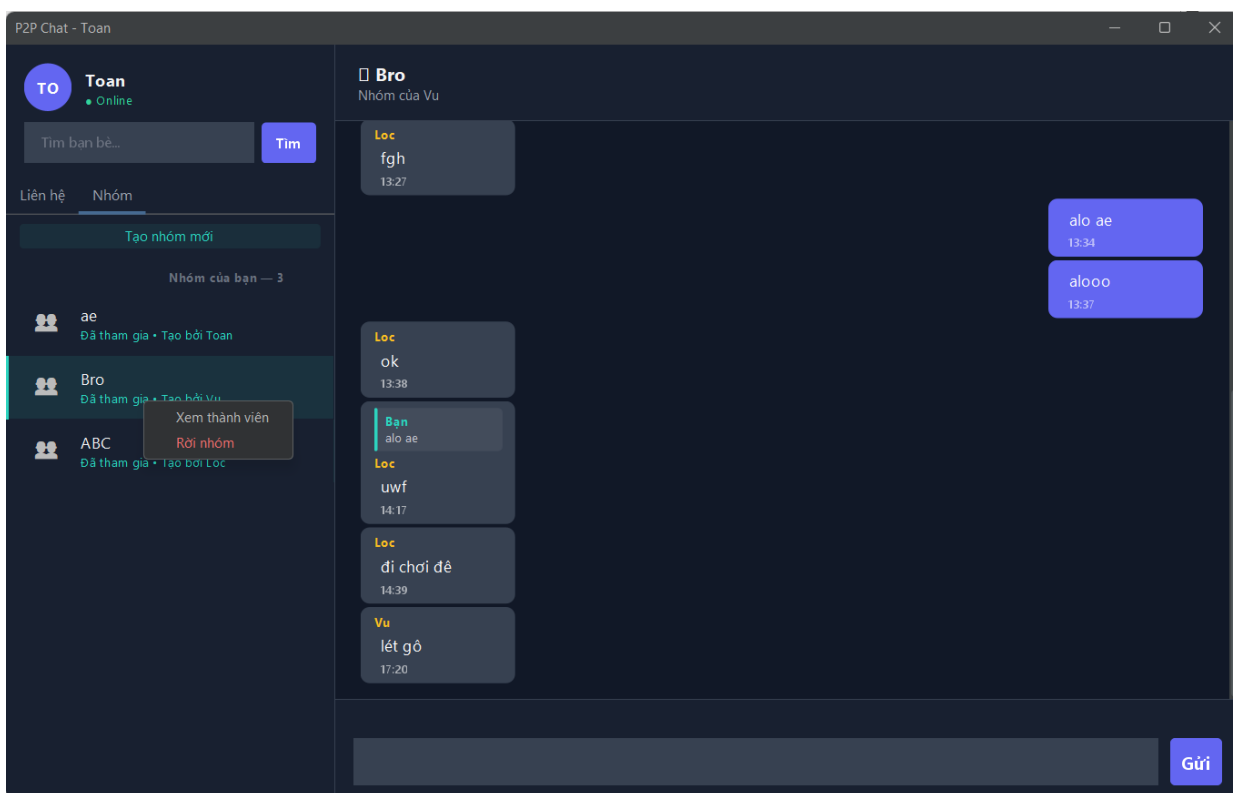
Người dùng cũng có thể vào các nhóm của các Admin khác



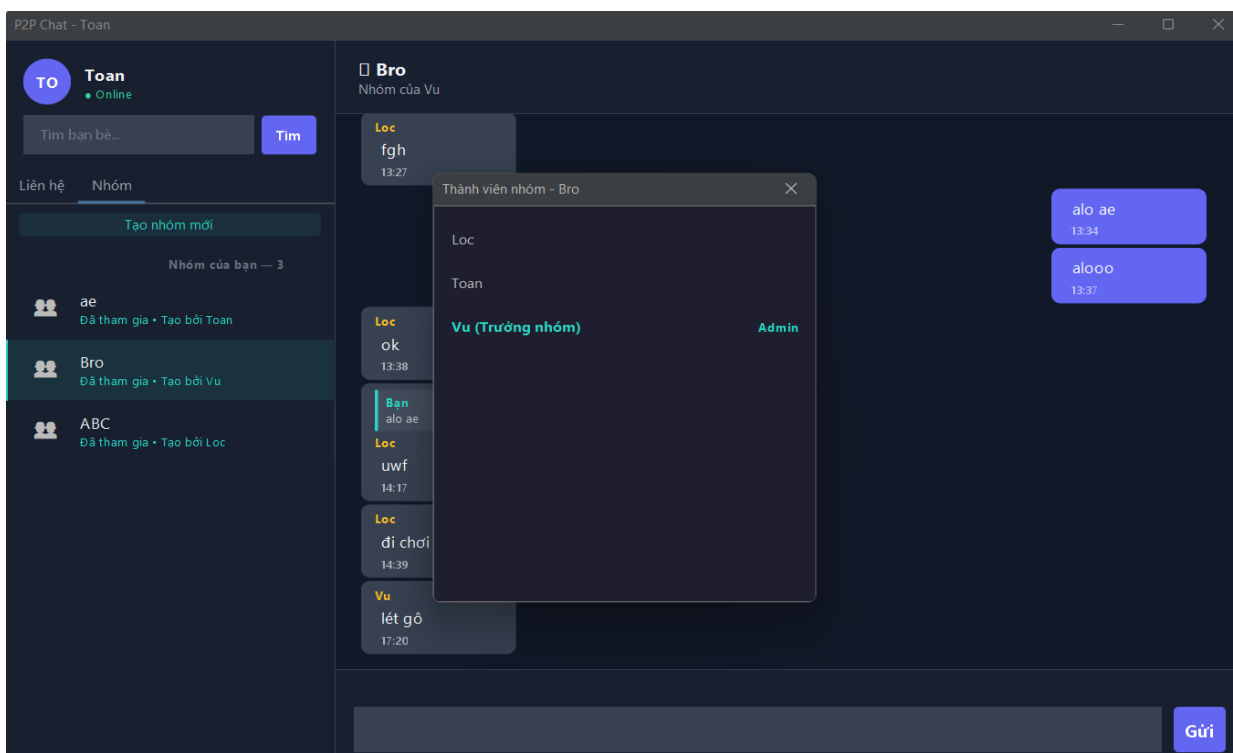
Giao diện tin nhắn trong 1 nhóm: Tin nhắn khi nhấn được gửi tới mọi người có tham gia nhóm. Tin nhắn cũng có thể trả lời, chuyển tiếp, thông báo tương tự chat cá nhân



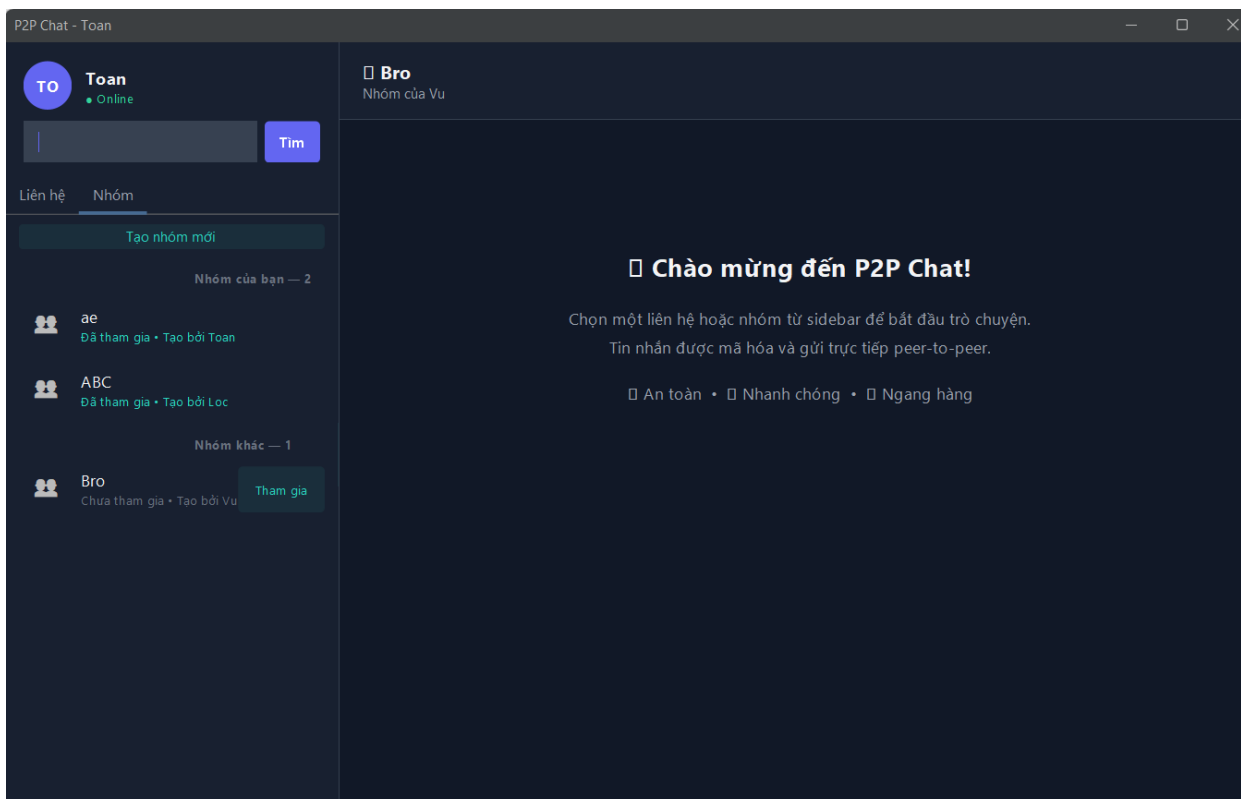
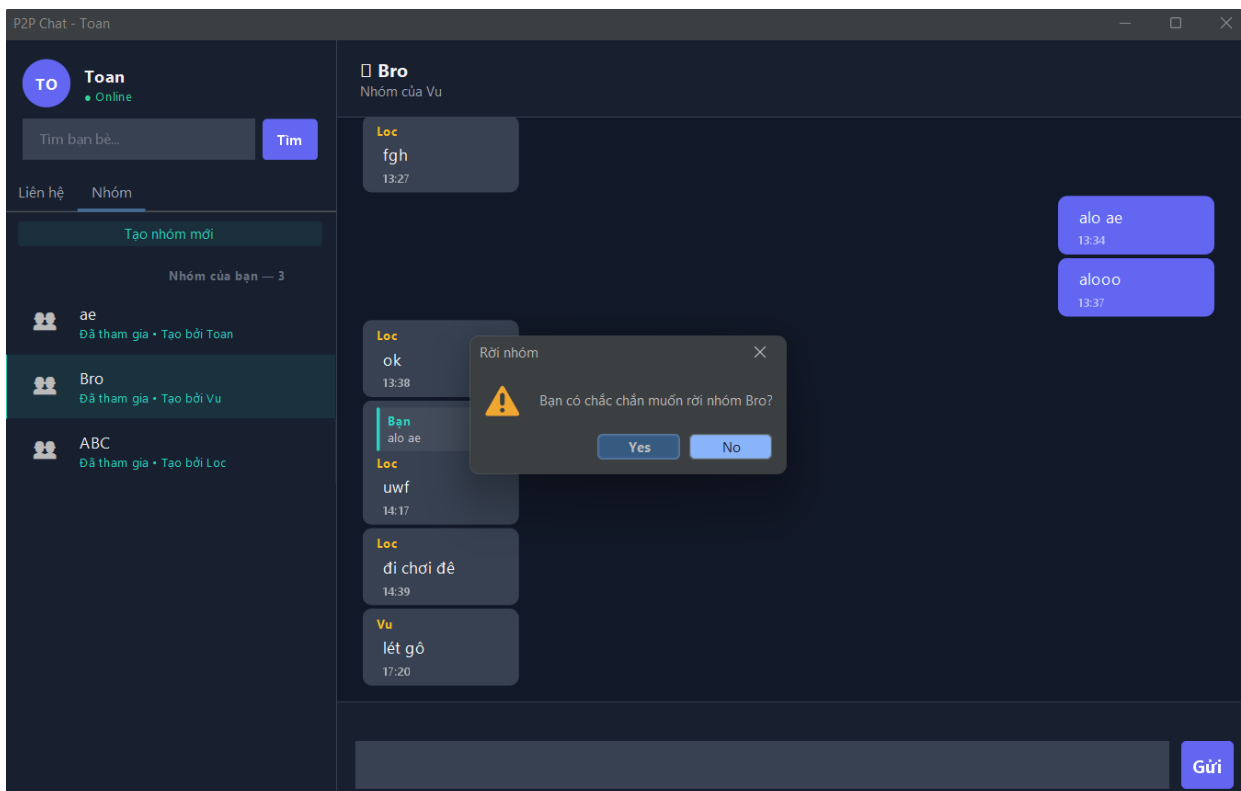
Người dùng có thể xem thành viên nhóm và rời nhóm



Danh sách thành viên nhóm

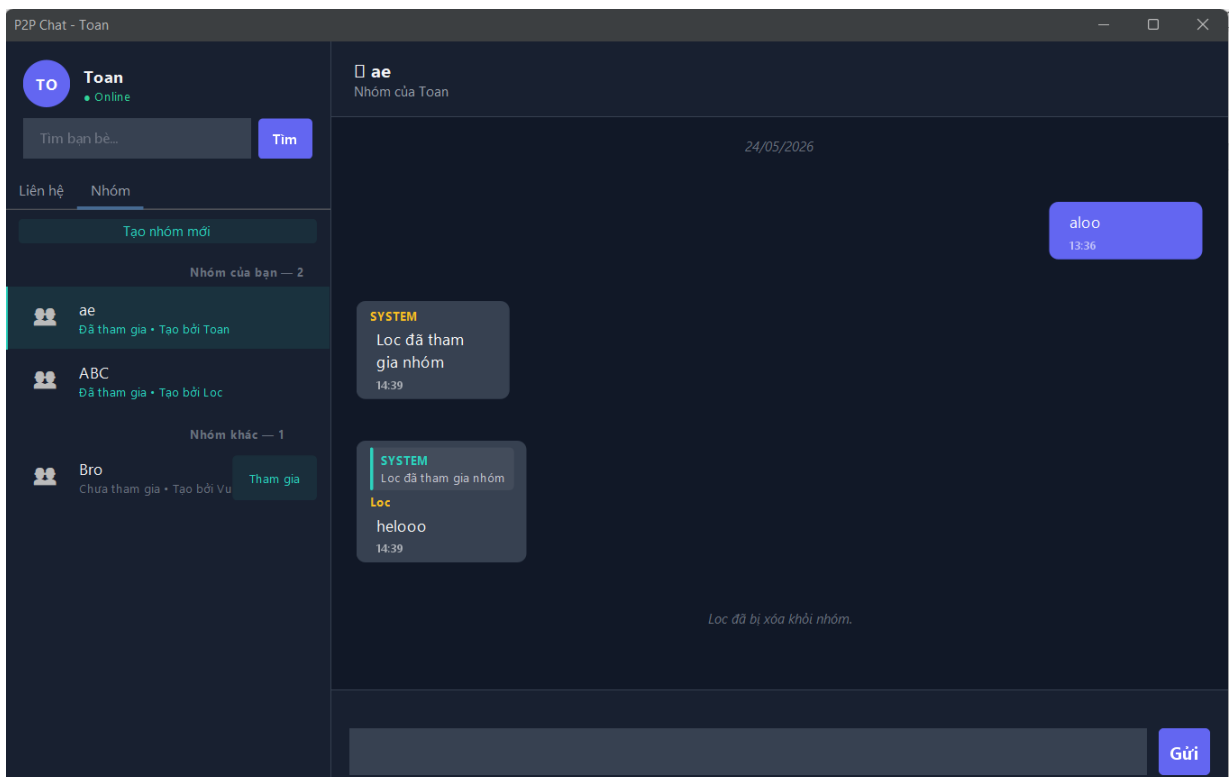
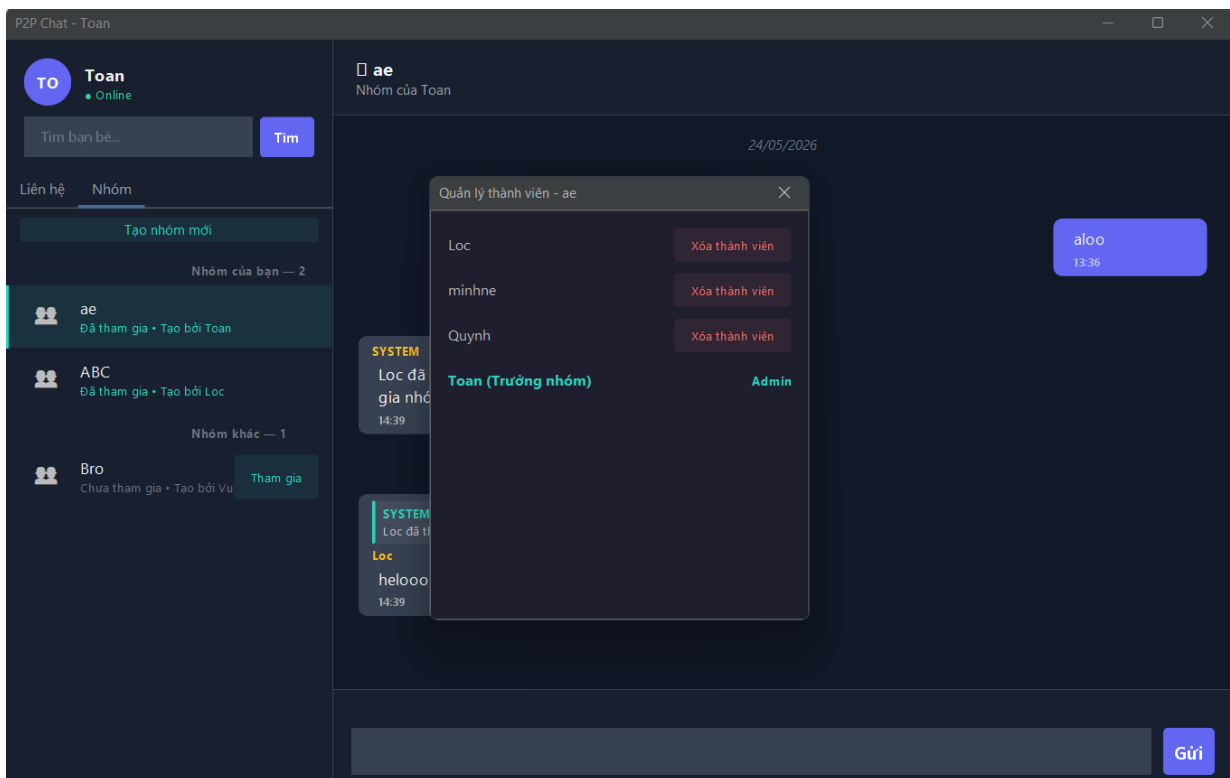


Khi rời khỏi nhóm

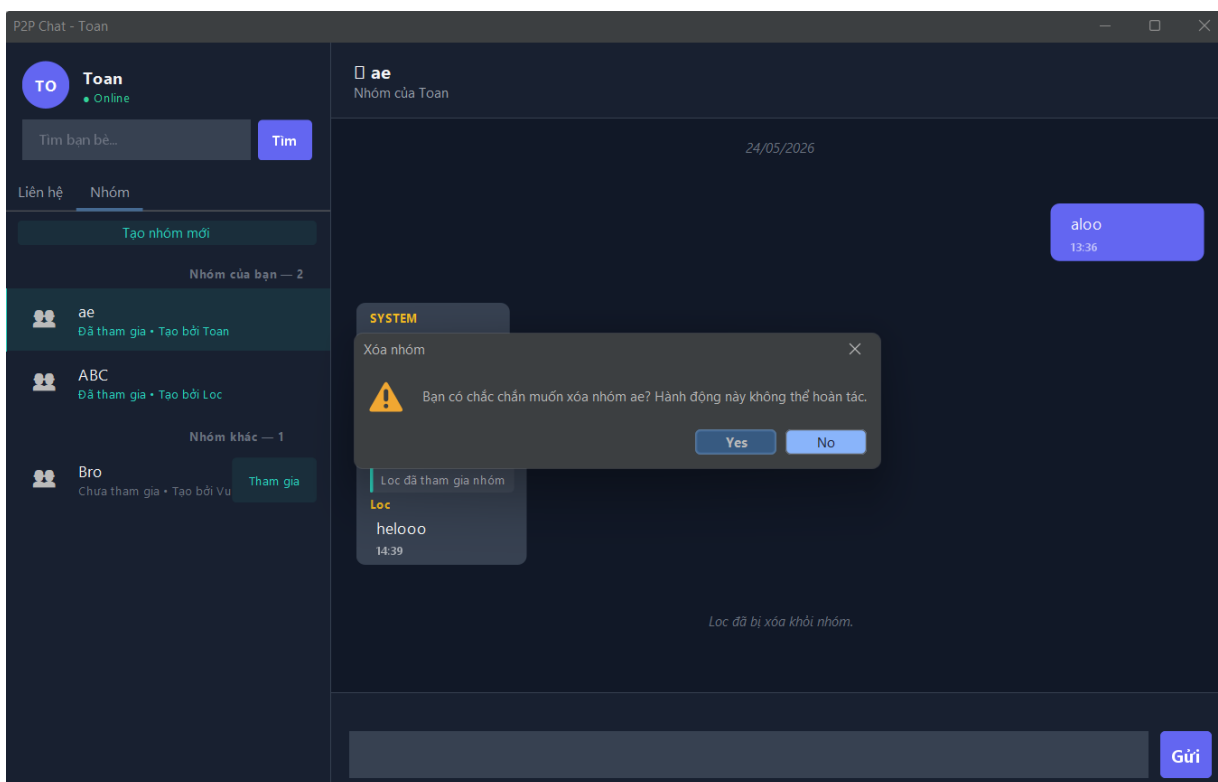


Giao diện khi xóa nhóm và xóa thành viên (Chức năng của Admin)

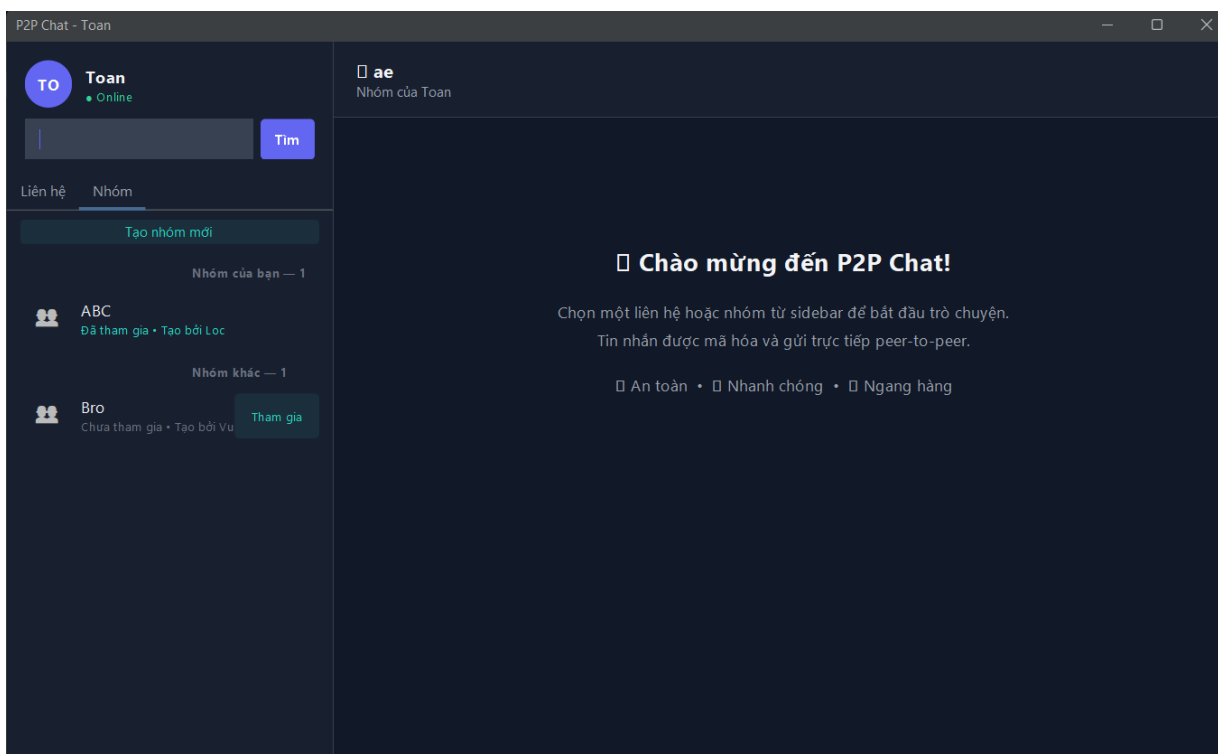
Danh sách hiện ra các thành viên và nút “Xóa thành viên”



Khi xóa nhóm hộp thoại sẽ hiện ra để xác nhận có muốn xóa hay không



Nhóm vừa xóa sẽ biến mất khỏi giao diện hệ thống



3.4 Kịch bản kiểm thử và sửa lỗi

Định nghĩa Peer A: Toan, Peer B: Loc

3.4.1 Mất kết nối đột ngột

Mô Tả Tình Huống: Kiểm tra cơ chế Heartbeat phát hiện mất kết nối đột ngột của Peer A mà không qua thủ tục Logout bình thường.

Các Bước Thực Hiện:

1. Đăng nhập Peer A và Peer B trên hệ thống.
2. Xác nhận cả hai cùng hiển thị trạng thái Online trên màn hình của nhau.
3. Tắt ứng dụng của Peer A đột ngột (nhấn tắt nóng cửa sổ Console/GUI hoặc ngắt mạng).
4. Quan sát cửa sổ Console của Server và trạng thái Peer A trên màn hình Peer B.

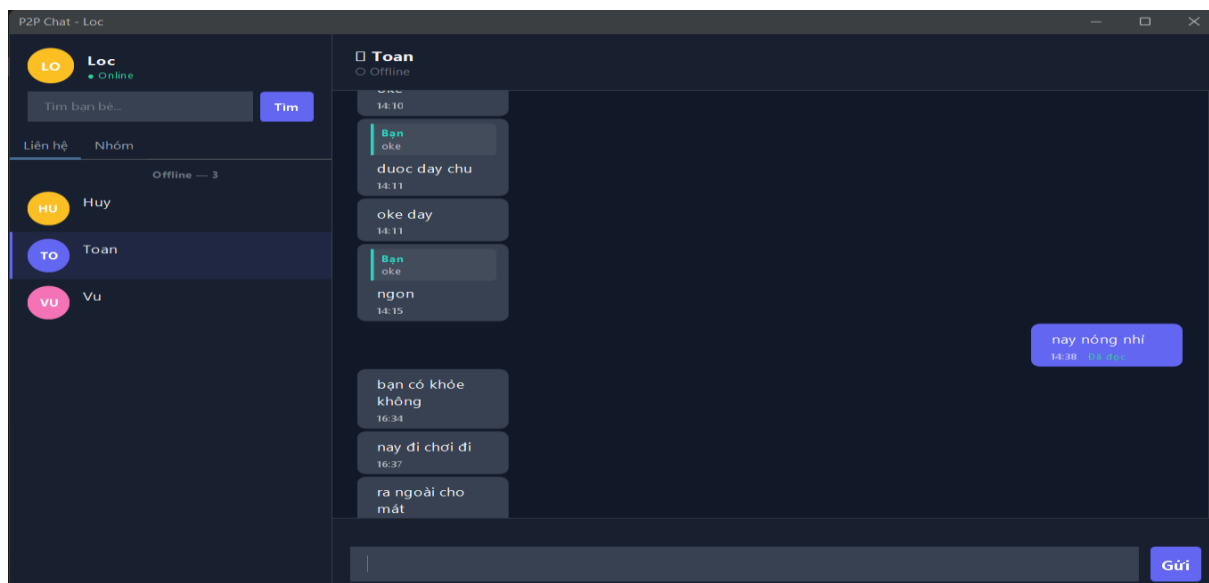
Kết Quả Mong Đợi:

- Cửa sổ Server xuất hiện dòng thông báo phát hiện Peer A mất kết nối.
- Trên giao diện của Peer B, biểu tượng trạng thái của Peer A chuyển từ chấm xanh Online sang chấm xám Offline

Kết quả thực hiện:

```
[Registry] Peer unregistered: Toan  
[Server] Peer timed out: Toan  
[Handler] Connection lost: Toan - Socket closed  
[Handler] User disconnected: Toan
```

Hình 3.4 Kết quả Console của Bootstrap Server



Hình 3.5 Kết quả trên Peer B

3.4.2 Gửi tin nhắn cho người dùng ngoại tuyến

Mô Tả Tình Huống: Xác minh thuật toán Store-and-Forward: lưu tin nhắn tạm thời trên Server khi người nhận offline và đồng bộ lại khi họ online.

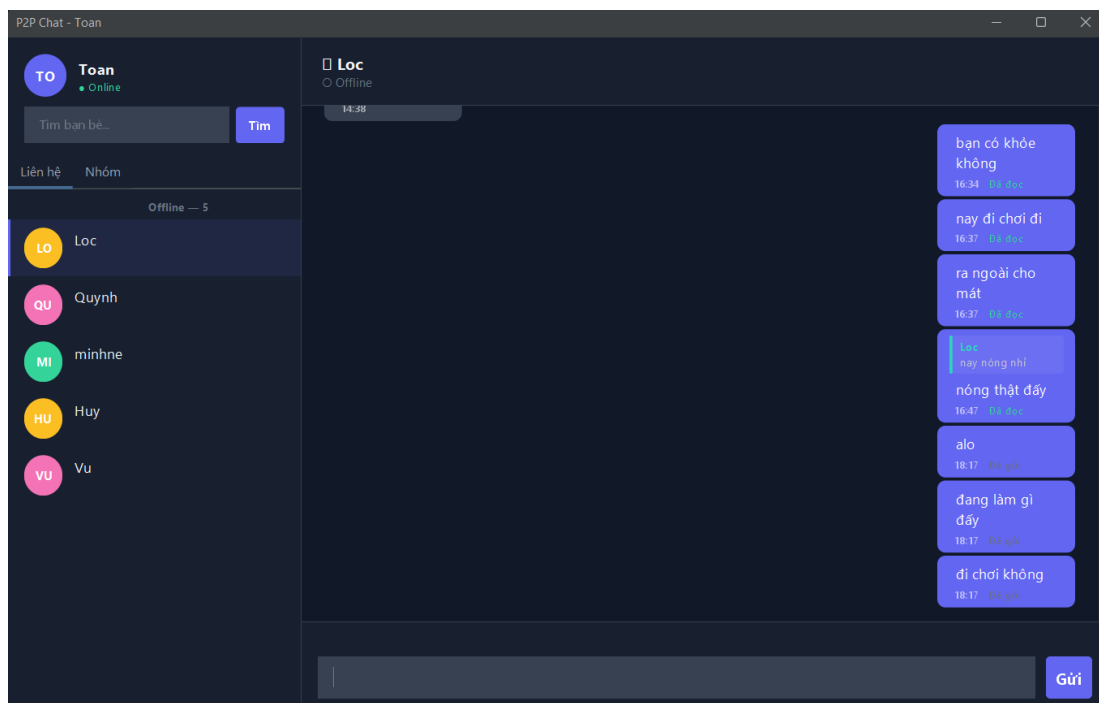
Các Bước Thực Hiện:

1. Cho Peer B đăng xuất hoặc tắt ứng dụng.
2. Peer A gửi 3 tin nhắn liên tiếp cho Peer B.
3. Kiểm tra bảng offline_messages trong database xem tin nhắn có được lưu trữ tạm thời không.
4. Khởi động lại ứng dụng Peer B và đăng nhập vào tài khoản.
5. Kiểm tra cửa sổ chat của Peer B.

Kết Quả Mong Đợi:

- Bảng offline_messages trong DB xuất hiện 3 dòng tin nhắn từ Peer A gửi tới Peer B.
- Ngay sau khi Peer B đăng nhập lại, 3 tin nhắn này tự động hiển thị trong khung chat của B.

Kết quả thực hiện:



Hình 3.6 Peer A gửi 3 tin nhắn cho Peer B (đang Offline)

Information: Table: **offline_messages**

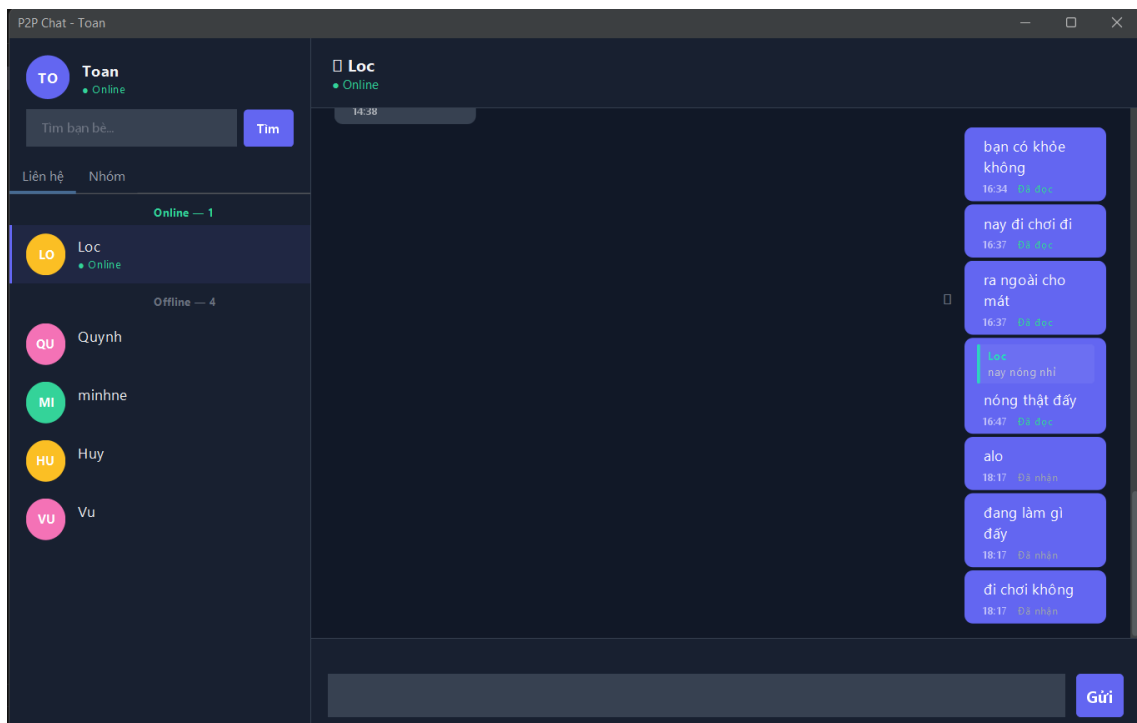
Columns:

Column Name	Data Type	Attributes
id	bigint	AI PK
message_id	varchar	
from_user	varchar	
to_user	varchar	
content	text	
message_type	varchar	
is_group_message	tinyint	
group_id	varchar	
timestamp	bigint	
created_at	times	

Result Grid

id	message_id	from_user	to_user	content	message_type	is_group_message	group_id	timestamp	created_at
57	a35088b9-46af-4e84-b998-36e244503218	Toan	Loc	alo	TEXT	0	HIDE	1779621460327	2026-05-24 18:17:~
58	a84bbf80-7905-4c6a-b442-4e9f999587a6	Toan	Loc	đang làm gì đấy	TEXT	0	HIDE	1779621465631	2026-05-24 18:17:~
59	0fa127da-1dfb-45f2-9903-9703096c91ca	Toan	Loc	đi chơi không	TEXT	0	HIDE	1779621469870	2026-05-24 18:17:~

Hình 3.7 Dữ liệu bảng offline_messages



Hình 3.8 Giao diện khi Peer B online (Trạng thái chuyển sang Đã nhận)

3.4.3 Đăng ký trùng tên tài khoản

Mô Tả Tình Huống: Kiểm tra tính toàn vẹn dữ liệu khi tạo tài khoản trùng với tên đã có sẵn trong cơ sở dữ liệu.

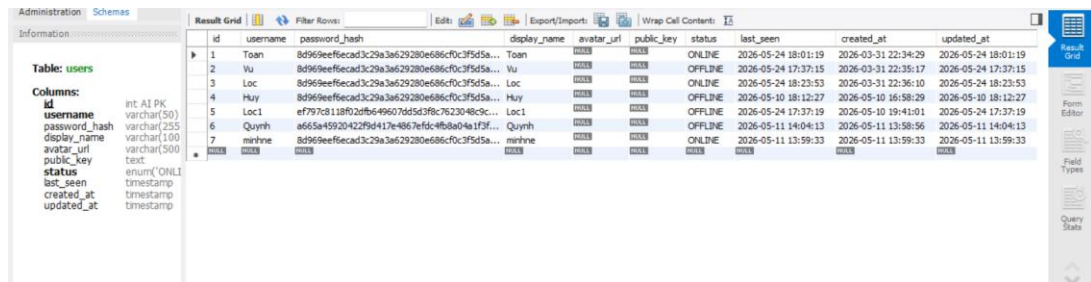
Các Bước Thực Hiện:

1. Mở cửa sổ đăng ký tài khoản mới.
2. Nhập username trùng khít với tài khoản đã tồn tại.
3. Nhập mật khẩu và nhấn Đăng ký.

Kết Quả Mong Đợi:

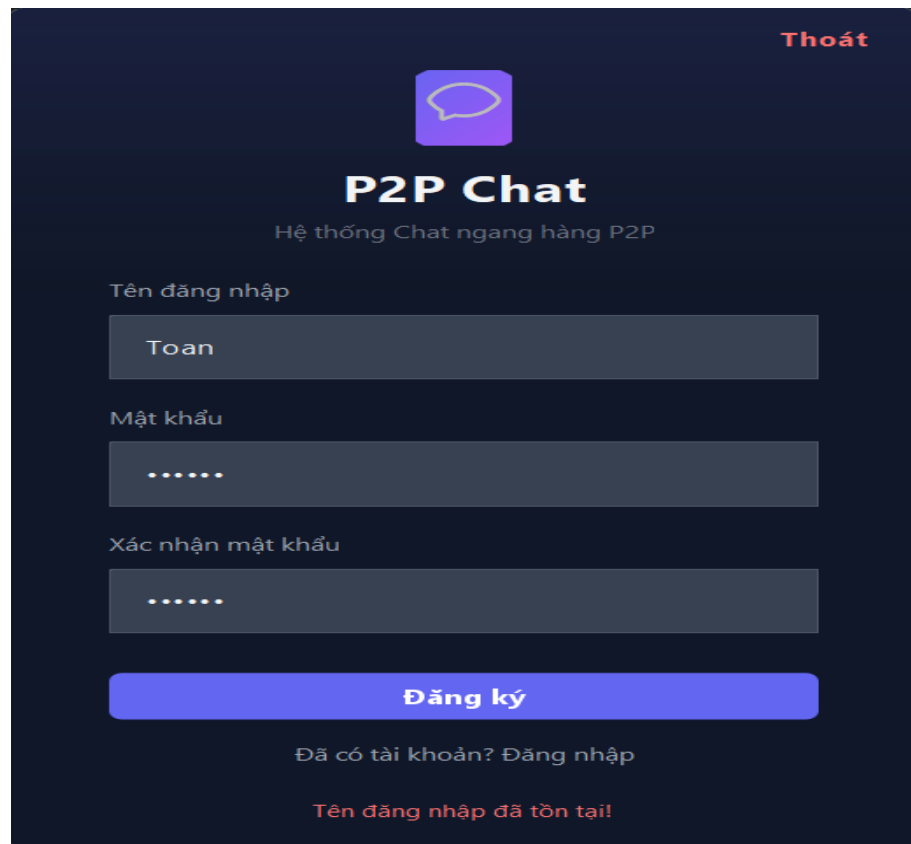
- Hệ thống từ chối đăng ký.
- Hiện thị thông báo lỗi chi tiết
- Không có tài khoản trùng lặp nào được tạo trong bảng users.

Kết quả thực hiện:

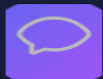


id	username	password_hash	display_name	avatar_url	public_key	status	last_seen	created_at	updated_at
1	Toan	8d969eeffecad3c29a3a629280e686cf0c3f5d5a...	Toan	avatar	public_key	ONLINE	2026-05-24 18:01:19	2026-03-31 22:34:29	2026-05-24 18:01:19
2	Vu	8d969eeffecad3c29a3a629280e686cf0c3f5d5a...	Vu	avatar	public_key	OFFLINE	2026-05-24 17:37:15	2026-03-31 22:35:17	2026-05-24 17:37:15
3	Loc	8d969eeffecad3c29a3a629280e686cf0c3f5d5a...	Loc	avatar	public_key	ONLINE	2026-05-24 18:23:53	2026-03-31 22:36:10	2026-05-24 18:23:53
4	Huy	8d969eeffecad3c29a3a629280e686cf0c3f5d5a...	Huy	avatar	public_key	OFFLINE	2026-05-10 18:12:27	2026-05-10 16:58:29	2026-05-10 18:12:27
5	Loc1	ef797c8118f02dfb649607d65d3fbc762304ec9c...	Loc1	avatar	public_key	OFFLINE	2026-05-24 17:37:19	2026-05-10 19:41:01	2026-05-24 17:37:19
6	Quỳnh	a665a4592b42299417e4867e7d6c4f8a9a1f5f...	Quỳnh	avatar	public_key	OFFLINE	2026-05-11 14:04:13	2026-05-11 13:58:56	2026-05-11 14:04:13
7	marine	8d969eeffecad3c29a3a629280e686cf0c3f5d5a...	marine	avatar	public_key	ONLINE	2026-05-11 13:59:33	2026-05-11 13:59:33	2026-05-11 13:59:33

Hình 3.9 Bảng dữ liệu users đã tồn tại



Thoát



P2P Chat

Hệ thống Chat ngang hàng P2P

Tên đăng nhập

Toan

Mật khẩu

.....

Xác nhận mật khẩu

.....

Đăng ký

Đã có tài khoản? Đăng nhập

Tên đăng nhập đã tồn tại!

Hình 3.10 Kết quả đăng ký bị lỗi khi trùng

3.5 Đánh giá và hướng phát triển

3.5.1. Ưu điểm của hệ thống

Tốc độ phản hồi cực nhanh và tối ưu tài nguyên: Nhờ sử dụng giao thức truyền tin cây TCP Socket trực tiếp kết hợp cấu trúc bản tin tối giản dạng JSON gọn nhẹ, thời gian trễ khi truyền gửi tin nhắn văn bản gần như bằng không. Việc đọc/ghi luồng nhị phân được tối ưu hóa giúp hệ thống hoạt động mượt mà ngay cả trong môi trường băng thông mạng nội bộ giới hạn.

Trải nghiệm người dùng UX/UI hiện đại: Ứng dụng tích hợp thư viện FlatLaf đem lại giao diện Dark theme chuyên nghiệp, đồng bộ tốt với các hệ điều hành Windows/macOS hiện đại. Các hiệu ứng chuyển đổi trạng thái gõ chữ, bong bóng trích dẫn trả lời tin nhắn và chỉ báo trạng thái tin nhắn (Đã gửi, Đã nhận, Đã đọc) hoạt động mượt mà, trực quan.

Cơ chế chịu lỗi bền bỉ. Hệ thống tự động phân loại và xử lý lỗi ngắt kết nối đột ngột nhờ luồng quét Heartbeat Monitor Thread chạy ngầm ở Server. Kết nối rác từ các máy trạm bị tắt nguồn đột ngột hoặc mất sóng LAN được dọn dẹp sạch sẽ chỉ sau tối đa 35 giây, tránh rò rỉ tài nguyên cổng Port Leaks trên máy chủ. Cơ chế Store-and-Forward (Lưu trữ và chuyển tiếp) giải quyết triệt để vấn đề mất mát thông tin khi đối phương ngoại tuyến (Offline), tự động đồng bộ lại tức thì ngay khi họ đăng nhập trở lại mà không cần sự can thiệp thủ công từ người dùng.

Tính toàn vẹn dữ liệu: Thiết kế cơ sở dữ liệu MySQL phân tách rõ ràng vai trò quản lý tài khoản, lịch sử hội thoại, danh sách bạn bè và thành viên nhóm. Các thao tác cập nhật trạng thái nhóm (Tạo/Rời/Xóa nhóm) được xử lý đồng bộ giao dịch chặt chẽ bằng cơ chế tự động Rollback nếu xảy ra xung đột hoặc lỗi truy vấn SQL.

3.5.2. Hạn chế thực tế của hệ thống

Bên cạnh những ưu điểm đạt được, hệ thống vẫn tồn tại một số điểm hạn chế kỹ thuật cần được nhìn nhận thực tế:

Hiệu năng cơ sở dữ liệu cục bộ: Việc mỗi Client tự duy trì một cơ sở dữ liệu MySQL cục bộ đòi hỏi máy trạm của người dùng phải cài đặt và cấu hình sẵn dịch vụ MySQL Server. Điều này gây khó khăn trong việc cài đặt nhanh ứng dụng cho người dùng phổ thông và làm tăng dung lượng tài nguyên RAM/CPU tiêu thụ trên thiết bị cá nhân.

Chưa tối ưu hóa luồng truyền tải dữ liệu lớn: Khi tiến hành truyền gửi tệp tin dung lượng lớn, việc đọc toàn bộ tệp tin vào bộ nhớ đệm RAM hoặc gửi gói tin có kích thước lớn qua luồng Socket thông thường dễ gây nghẽn băng thông kết nối hoặc lỗi tràn bộ nhớ nếu không được phân đoạn và kiểm soát luồng chặt chẽ.

3.5.3. Hướng phát triển tương lai

Để hoàn thiện hệ thống trở thành một sản phẩm Chat P2P thương mại hóa và an toàn hơn, các hướng nghiên cứu và nâng cấp tiếp theo bao gồm:

Hiện thực hóa tính năng truyền tệp tin trực tiếp phân đoạn: Tận dụng cấu trúc dữ liệu truyền file phân đoạn đã được định nghĩa sẵn trong hệ thống `FILE_CHUNK_SIZE = 64KB`. Phát triển luồng truyền tệp trực tiếp Direct P2P File Transfer bằng cách thiết lập một luồng Socket TCP riêng biệt trực tiếp giữa hai máy trạm Peer-to-Peer mà không đi qua Bootstrap Server, giúp giải phóng băng thông cho Server chính. Tệp tin sẽ được băm nhỏ thành các khối 64KB, gửi tuần tự kèm mã băm SHA-256 ở cuối mỗi khối để kiểm tra tính toàn vẹn và ghép lại hoàn chỉnh ở máy nhận.

Kích hoạt mã hóa đầu cuối toàn diện End-to-End Encryption - E2EE: Tích hợp sâu thư viện mã hóa đã cấu hình sẵn trong `Constants.java` bao gồm cặp khóa công khai/bí mật RSA 2048-bit và thuật toán mã hóa đối xứng hiệu năng cao AES-GCM 256-bit. Thiết lập cơ chế trao đổi khóa bảo mật Diffie-Hellman qua Socket. Nội dung tin nhắn và tệp tin sẽ được mã hóa ngay tại thiết bị người gửi Client A và chỉ có thể được giải mã tại thiết bị người nhận Client B. Bootstrap Server trung gian hoàn toàn không thể đọc được nội dung truyền tải.

Tối ưu hóa cài đặt và cơ sở dữ liệu cục bộ: Thay thế hệ quản trị cơ sở dữ liệu MySQL cồng kềnh ở phía Client bằng hệ cơ sở dữ liệu nhúng siêu nhẹ như SQLite hoặc H2 Database. Client sẽ chạy trực tiếp file JAR mà không cần cài đặt thêm bất kỳ phần mềm cơ sở dữ liệu nào khác, tăng tính di động và dễ dàng triển khai.

Nâng cấp khả năng vượt tường lửa NAT Traversal: Triển khai giao thức UDP Hole Punching để giúp hai Peer đứng sau các lớp NAT khác nhau tự khám phá IP công cộng của nhau và tự thiết lập đường truyền trực tiếp mà không cần Server làm trung gian chuyển tiếp tin nhắn.

KẾT LUẬN

Đề tài “Xây dựng hệ thống chat ngang hàng P2P” đã được triển khai thành công với việc hoàn thiện hầu hết các chức năng cốt lõi của một ứng dụng giao tiếp phân tán hiện đại. Qua quá trình phát triển và thử nghiệm, phần mềm đã chứng minh khả năng đáp ứng tốt nhu cầu trao đổi thông tin thời gian thực, đồng thời đảm bảo tính ổn định cao nhờ thiết kế mô hình mạng P2P kết hợp linh hoạt giữa vai trò định danh của máy chủ trung tâm và kết nối cục bộ.

Hệ thống đã tích hợp thành công các chức năng chính như: nhắn tin cá nhân, trò chuyện nhóm, quản lý danh sách bạn bè, cùng cơ chế lưu trữ và chuyển tiếp (Store-and-Forward) giúp nhận gửi tin nhắn ngay cả khi đối phương ngoại tuyến. Trải nghiệm nhắn tin được tối ưu hóa mượt mà nhờ việc đồng bộ hóa tức thì các trạng thái đang gõ chữ, trạng thái đã đọc, cũng như các thao tác tương tác sâu như trích dẫn trả lời và chuyển tiếp tin nhắn. Bên cạnh đó, hệ thống cũng giải quyết tốt các bài toán mạng phân tán thực tế bằng cách tự động phát hiện và dọn dẹp các kết nối rác thông qua cơ chế Heartbeat, đồng thời mang lại trải nghiệm cài đặt dễ dàng qua tệp tin đóng gói độc lập Fat JAR cùng giao diện Dark theme chuyên nghiệp.

Bên cạnh những thành tựu đã đạt được, hệ thống vẫn còn một số hạn chế nhất định cần được khắc phục để hoàn thiện hơn. Đầu tiên phải kể đến việc chưa kích hoạt toàn diện chuẩn mã hóa đầu cuối End-to-End Encryption bằng bộ khóa RSA và AES nhằm bảo mật tuyệt đối dữ liệu trên đường truyền. Về mặt tính năng, dự án chưa triển khai được luồng chia sẻ tệp tin phân đoạn trực tiếp File Chunk Transfer để giảm tải cho máy chủ. Cuối cùng, để đạt được hiệu năng ngang hàng đúng nghĩa, hệ thống cần được nâng cấp thêm cơ chế vượt tường lửa NAT Hole Punching giúp các máy trạm ở các phân mạng khác nhau có thể tự động tìm thấy nhau và giao tiếp trực tiếp mà không cần phải chuyển tiếp dữ liệu quá nhiều qua Bootstrap Server.

TÀI LIỆU THAM KHẢO

- [1] **Oracle Corporation**, "*Java Platform, Standard Edition 17 API Specification*", Java Documentation. Địa chỉ: <https://docs.oracle.com/en/java/javase/17/docs/api/>. (Tài liệu tham khảo về kiến trúc lập trình mạng *Java.net.Socket*, luồng I/O và *Multi-threading*).
- [2] **Oracle Corporation**, "*MySQL 8.0 Reference Manual*", MySQL Documentation. [Trực tuyến]. Địa chỉ: <https://dev.mysql.com/doc/refman/8.0/en/>. (Tài liệu cấu hình cơ sở dữ liệu, cú pháp truy vấn SQL và quản lý giao dịch *Transaction*).
- [3] **Kurose, J. F., & Ross, K. W.**, "*Computer Networking: A Top-Down Approach (8th Edition)*", Pearson. (Sách tham khảo lý thuyết cơ sở về nguyên lý mạng ngang hàng P2P, mô hình Client-Server và giao thức truyền vận TCP/IP).
- [4] **FormDev Software GmbH**, "*FlatLaf - Flat Look and Feel for Java Swing*", 2023. [Trực tuyến]. Địa chỉ: <https://www.formdev.com/flatlaf/>. (Tài liệu hướng dẫn triển khai giao diện người dùng Dark theme hiện đại cho ứng dụng Desktop).
- [5] **Google Inc.**, "*Gson User Guide*", GitHub Repository. [Trực tuyến]. Địa chỉ: <https://github.com/google/gson/blob/master/UserGuide.md>. (Tài liệu hướng dẫn kỹ thuật *Serialize* và *Deserialize* đối tượng Java thành chuỗi JSON phục vụ việc đóng gói bản tin).
- [6] **Brett Wooldridge**, "*HikariCP - A solid, high-performance, JDBC connection pool*", GitHub Repository. [Trực tuyến]. Địa chỉ: <https://github.com/brettwooldridge/HikariCP>. (Tài liệu giải pháp quản lý nhóm kết nối cơ sở dữ liệu (*Connection Pooling*) giúp tối ưu hiệu năng).
- [7] **The Apache Software Foundation**, "*Apache Maven Project*", Maven Documentation. [Trực tuyến]. Địa chỉ: <https://maven.apache.org/>. (Tài liệu tham khảo về cấu trúc quản lý thư viện (*POM*) và quy trình biên dịch, đóng gói ứng dụng *Fat JAR*).